

1. Numpy及其基本使用

Numpy是我们接触科学计算的第一个工具包，主要是用来解决向量与矩阵的运算。它把矩阵抽象为二维数组，在二维数组上定义了一系列矩阵算术运算的基本操作。它的底层是通过C语言实现，相比于原生Python自带的list、dict等方法有更好的效率。在这一章当中，我们将会学习到的内容包括：

- [1.1 Numpy中的数组](#)
- [1.2 Numpy数组的运算](#)
- [1.3 Numpy与线性代数](#)
- [1.4 Numpy与多项式计算](#)

1.1 Numpy中的数组

1.1.1 Numpy的数据格式

numpy中所支持的数据类型其实相比原生Python，它更接近于C。光是整数类型，就分了int32, int64, uint32, uint64等好几个类型。包括浮点数也分为float32和float64。在字长和类型上进行了更好的统一，方便numpy与C的无缝衔接。

而我们之所以引入Numpy，一个很重要的原因就是Numpy数组可以很好地模拟向量与矩阵运算。我们可以通过numpy.array创建一个数组，例如：

```
import numpy as np
x=np.array([1,2,3,4,5])
```

这样我们可以创建一个numpy数组。但是numpy的数组和Python原生的列表数据结构并不相同，它不能直接append、pop等。并且在numpy数组中，元素的数据类型必须要统一，不允许多个不同类型数据存放在同一个数组当中。那numpy数组具备怎样的特性呢？在后面我们会看到，numpy.array很重要的特性就是它可以模拟线性代数当中的各种运算。

与二重列表类似，numpy也支持二重数组以及更高维度的多重数组。并且numpy约定，每个子数组的长度必须保持平齐。

那我们如何指定一个数组的数据类型呢？我们可以通过这样的方法：

```
x=np.array([1,2,3],dtype=np.int32)
```

这样，我们创建了一个数据类型为numpy.int32类型的数组。

numpy数组与其他数据类型之间也可以进行方便的数据转换。例如，列表、元组可以转换为numpy.array，而numpy.array也可以转换为列表等数据结构。例如，我们有两种方法把上述数组修改为Python列表类型：

```
# 方法1：直接强制类型转换：
x=list(x)
# 方法2：使用tolist方法
x=x.tolist()
```

那么如果想要修改numpy数组当中的数据元素类型，我们可以通过astype指定。

```
x.astype(np.float32)
```

在numpy当中，有一些快速创建数组的方法，例如：

```
1. numpy.zeros((m,n))
```

这个方法被用于创建一个形状为 (m, n) 的全0数组。例如：

案例：

```
import numpy as np

# 创建一个2x3的零矩阵
zero_matrix = np.zeros((2, 3))
print(zero_matrix)

# 输出：
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

```
2. numpy.ones((m,n))
```

这个方法被用于创建一个形状为 (m, n) 的全1数组。例如：

案例：

```
import numpy as np

# 创建一个2x3的全1矩阵
ones_matrix = np.ones((2, 3))
print(ones_matrix)

# 输出:
# [[1. 1. 1.]
#  [1. 1. 1.]]
```

注意，这两个方法传进去的参数都是元组！要记得打两次括号！当然，如果只传入一个数字也是可以的，它会默认给你创建一个n*n的方阵。

3. `numpy.full((m,n), value)`

创建一个形状为 (m, n) 的数组，并用给定的值 value 填充数组中的所有元素。

案例：

```
import numpy as np

# 创建一个2x3的数组，所有元素都是42
full_matrix = np.full((2, 3), 42)
print(full_matrix)

# 输出:
# [[42 42 42]
#  [42 42 42]]
```

4. `numpy.arange(start, stop, step)`

这个方法可以生成一个等差数列，从 start （包括）到 stop （不包括），步长为 step 。如果省略 step ，则默认为1。

案例：

```
import numpy as np

# 生成从0到5（不包括5）的数组，步长为1
arange_array = np.arange(0, 5)
print(arange_array)
# 输出：
# [0 1 2 3 4]

# 生成从0到10的数组，步长为2
arange_step_array = np.arange(0, 11, 2)
print(arange_step_array)
# 输出：
# [ 0  2  4  6  8 10]
```

它和Python自带的range函数用法类似，但是np.arange返回的是一个实打实的向量，而Python自带的range嘛……严格意义上来讲，应该叫做一个迭代器。

5. numpy.linspace(start, stop, num)

在指定的间隔内返回均匀间隔的数字，即生成一个等差数列，但这里是基于元素数量 num 来均匀分配间隔。

案例：

```
import numpy as np

# 生成从0到10的5个等间隔的数
linspace_array = np.linspace(0, 10, 5)
print(linspace_array)
# 输出：
# [ 0.   2.5   5.   7.5 10. ]
```

6. numpy.diag(array, k=0)

如果 array 是一维数组，则返回一个以 array 为对角线的方阵（k=0 表示主对角线）。如果 array 是二维数组，则返回其指定对角线上的元素。

案例：

```
import numpy as np

# 使用一维数组创建对角矩阵
diag_matrix = np.diag([1, 2, 3])
print(diag_matrix)

# 输出:
# [[1 0 0]
#  [0 2 0]
#  [0 0 3]]

# 使用二维数组提取对角线元素
diag_elements = np.diag(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
print(diag_elements)

# 输出:
# [1 5 9]
```

7. `numpy.eye(N, M=None, k=0, dtype=float, order='C')`

这个方法返回一个二维数组（矩阵），其中对角线元素为1，其余元素为0。这也正是单位矩阵的定义。如果 `M` 为 `None`，则与 `N` 相同，即生成一个 `N x N` 的单位矩阵。`k` 指定了对角线的索引，`k=0` 表示主对角线。

这里是 `numpy.eye()` 的完整案例：

```
import numpy as np

# 创建一个3x3的单位矩阵
eye_matrix = np.eye(3)
print(eye_matrix)
# 输出:
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

# 创建一个3x4的矩阵，其主对角线元素为1，其余为0
eye_matrix_3x4 = np.eye(3, 4)
print(eye_matrix_3x4)
# 输出:
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]]

# 创建一个3x3的矩阵，其对角线向上偏移一个单位（次对角线为1）
eye_matrix_k1 = np.eye(3, k=1)
print(eye_matrix_k1)
# 输出:
# [[0. 1. 0.]
#  [0. 0. 1.]
#  [0. 0. 0.]]

# 创建一个3x3的矩阵，其对角线向下偏移一个单位（超对角线为1）
eye_matrix_k_1 = np.eye(3, k=-1)
print(eye_matrix_k_1)
# 输出:
# [[0. 0. 0.]
#  [1. 0. 0.]
#  [0. 1. 0.]]
```

在这个案例中，`np.eye(3)` 生成了一个3x3的单位矩阵，其中主对角线上的元素为1，其余元素为0。`np.eye(3, 4)` 创建了一个3x4的矩阵，其形状不是方阵，但主对角线上的元素仍然为1。通过 `k` 参数，你可以控制对角线的位置，`k=1` 表示次对角线（对角线向上偏移一个单位）为1，而 `k=-1` 表示超对角线（对角线向下偏移一个单位）为1。

通过这些方法，可以快速创建numpy数组。

1.1.2 Numpy数组的变换

1. 数组的转置

数组（或者说矩阵）的转置是指将数组的行和列进行互换。对于二维数组（矩阵）来说，转置意味着将原矩阵的行列互换，即原矩阵的第*i*行第*j*列的元素会变为转置矩阵的第*j*行第*i*列的元素。NumPy提供了非常简便的方法来实现数组的转置，主要通过 `.T` 属性或者 `numpy.transpose()` 函数。使用 `.T` 属性这是最常用的方法，因为它既简洁又直观。

```
import numpy as np

# 创建一个二维数组
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

# 使用.T属性进行转置
arr_T = arr.T

print(arr_T)
```

输出将会是：

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

`numpy.transpose()` 函数也可以用来转置数组，其用法与 `.T` 属性类似，但提供了更多的灵活性，比如可以指定转置的轴。

```
# 使用numpy.transpose()函数进行转置
arr_transposed = np.transpose(arr)

print(arr_transposed)
```

输出与上面使用 `.T` 属性的例子相同。

在更复杂的数组中（比如三维或更高维度的数组），`.T` 属性只会交换前两个轴，而 `numpy.transpose()` 函数允许你通过 `axes` 参数来指定任意轴的转置顺序。例如，在三维数组中，你可以通过指定 `axes` 参数来定义三个轴之间的转置关系。

另外，向量也是可以进行转置的。如果你只用一层中括号表示向量，那么它转置与否无所谓。但如果是用两层括号表示向量，也就是二维数组的形式，那么向量就会有行向量和列向量之分。

2. 数组的变形

在NumPy中，改变数组的形状通常是通过 `reshape` 函数来实现的。`reshape` 函数允许你在不改变数组数据的情况下，重新排列数组的元素以形成新的形状。这要求新形状的总元素数量与原始数组的总元素数量相同。

`reshape` 函数的基本语法如下：

```
numpy.reshape(a, newshape)
```

- `a` 是要改变的原始数组。
- `newshape` 是新的形状，可以是整数或整数元组的形式。

如果 `newshape` 是一个整数，那么它会被视为新的形状中的一维长度，NumPy会自动计算出其他维度的长度。如果 `newshape` 是整数元组，那么它应该明确指定每个维度的大小。

假设我们有一个一维数组，我们想要将其重塑为一个二维数组：

```
import numpy as np

# 创建一个一维数组
arr = np.arange(6) # array([0, 1, 2, 3, 4, 5])

# 使用reshape重塑为2x3的二维数组
arr_resaped = arr.reshape((2, 3))

print(arr_resaped)

# 输出：
# [[0 1 2]
#  [3 4 5]]
```

在这个例子中，`reshape` 将原始的一维数组 `arr` 重塑为一个2行3列的二维数组 `arr_resaped`。

这里需要注意两点：

- 原始数组的总元素数量必须与新形状的总元素数量相匹配，否则 `reshape` 会抛出异常。
- 使用 `-1` 作为 `newshape` 中的一个维度可以让NumPy自动计算该维度的大小。这在你不知道某个维度具体大小但知道其他维度大小时非常有用。

例如，如果你有一个包含6个元素的一维数组，并且你想要重塑为一个3行的二维数组，但不确定每行应该有多少个元素，你可以这样做：

```
arr_resaped = arr.reshape((3, -1))
print(arr_resaped)
# 输出：
# [[0 1]
#  [2 3]
#  [4 5]]
```

在这个例子中，NumPy自动计算出第二维（列）应该是2，以便总元素数量保持为6。

另外，`reshape` 函数返回一个新的数组对象，该对象具有指定的形状，但包含与原始数组相同的数据。原始数组不会被修改。

当然，当你想要将一个多维数组转换为一维数组时，`np.flatten()` 是一个非常有用的函数。`np.flatten()` 会返回数组的一份拷贝，该拷贝被转换成了一维数组，且保持原始数据的顺序不变。

```
numpy.flatten(a, order='C')
```

```
import numpy as np

# 创建一个二维数组
arr = np.array([[1, 2, 3], [4, 5, 6]])

# 使用flatten转换为一维数组
arr_flat = arr.flatten() # 注意：这里也可以直接使用 np.flatten(arr)，但使用数组对象的方法更常见

print(arr_flat)
# 输出：
# [1 2 3 4 5 6]
```

在这个例子中，`flatten()` 方法将二维数组 `arr` 转换成了一个一维数组 `arr_flat`。

虽然 `flatten()` 可以很方便地将多维数组转换为一维数组，但使用 `reshape(-1)` 也是达到同样目的的一种常见且高效的方法。`reshape(-1)` 通过指定 `-1` 来自动计算新形状中唯一维度的大小，这通常用于将数组展平。

```
arr_reshaped = arr.reshape(-1)
print(arr_reshaped)
# 输出与 arr_flat 相同
# [1 2 3 4 5 6]
```

在大多数情况下，`reshape(-1)` 和 `flatten()` 在功能上是等价的，但 `reshape(-1)` 可能在性能上更优，因为它不需要创建数组的额外副本。在需要明确保持数组连续性的场景中，使用 `reshape(-1)` 可能更为合适。

3. 数组的堆叠

在NumPy中，实现数组拼接主要有以下几种方法：

1. `numpy.concatenate()`：

这是最常用的数组拼接函数之一，它可以沿着指定的轴将多个数组序列连接起来。默认情况下，`concatenate` 函数会沿着第一个轴（`axis=0`）进行拼接，但你可以通过 `axis` 参数来指定其他轴。

```

import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])

# 默认沿着第一个轴 (axis=0) 拼接
c = np.concatenate((a, b), axis=0)
print(c)
# 输出:
# [[1 2]
#  [3 4]
#  [5 6]]

# 沿着第二个轴 (axis=1) 拼接
d = np.concatenate((a, b.T), axis=1) # 注意b需要转置以匹配a的形状
print(d)
# 输出:
# [[1 2 5]
#  [3 4 6]]

```

2. `numpy.stack()` :

`stack` 函数将一系列数组沿着新的轴进行堆叠。这与 `concatenate` 不同，因为 `stack` 会创建一个新的维度。

```

a = np.array([1, 2])
b = np.array([3, 4])

# 沿着新的轴 (axis=0) 堆叠
c = np.stack((a, b), axis=0)
print(c)
# 输出:
# [[1 2]
#  [3 4]]

# 沿着新的轴 (axis=1) 堆叠
d = np.stack((a, b), axis=1)
print(d)
# 输出:
# [[1 3]
#  [2 4]]

```

3. `numpy.vstack()` 和 `numpy.hstack()` :

这两个函数是 `concatenate` 的便捷包装，分别用于垂直（沿着第一个轴）和水平（沿着第二个轴）堆叠数组。

- `vstack` （垂直堆叠）：

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.vstack((a, b))
print(c)
# 输出:
# [[1 2]
#  [3 4]]
```

- `hstack` （水平堆叠）：

```
a = np.array([[1], [2]])
b = np.array([[3], [4]])
c = np.hstack((a, b))
print(c)
# 输出:
# [[1 3]
#  [2 4]]
```

4. `numpy.column_stack()` 和 `numpy.row_stack()` :

这两个函数与 `hstack` 和 `vstack` 类似，但它们接受一维数组作为输入，并将它们视为列或行。

- `column_stack` （列堆叠）：

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.column_stack((a, b))
print(c)
# 输出:
# [[1 3]
#  [2 4]]
```

- `row_stack` （行堆叠）：

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.row_stack((a, b))
print(c)
# 输出:
# [[1 2]
#  [3 4]]
```

注意: `row_stack` 实际上就是 `vstack` 的别名。

在选择这些方法时,重要的是要考虑你的具体需求,比如你希望沿着哪个轴进行拼接,以及你的输入数组的形状。

4. 元素追加与删除

在NumPy中,数组(尤其是ndarray对象)通常被设计为固定大小的,这意呀着它们不像Python的列表(list)那样可以方便地追加或删除元素。然而,NumPy提供了一些方法来间接实现这些操作,但通常这些方法会涉及到创建新的数组,而不是在原地修改原始数组。

5. 追加元素

要在NumPy数组中追加元素,你可以使

用 `numpy.concatenate()`、`numpy.append()` 或 `numpy.vstack()` (对于二维数组作为行追加) 等函数。不过,需要注意的是, `numpy.append()` 实际上是一个包装了 `numpy.concatenate()` 的便捷函数,但它通常比直接使用 `concatenate()` 要慢,因为它会创建更多的临时数组。

- 使用 `numpy.concatenate()`

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5])

# 沿着第一个轴 (axis=0) 追加
c = np.concatenate((a, b))
print(c) # 输出: [1 2 3 4 5]
```

- 使用 `numpy.append()`

```
a = np.array([1, 2, 3])

# 追加单个元素
b = np.append(a, 4)
print(b) # 输出: [1 2 3 4]

# 追加数组
c = np.append(a, [4, 5])
print(c) # 输出: [1 2 3 4 5]
```

在NumPy中删除元素通常意味着创建一个新的数组，它包含了除了要删除的元素之外的所有元素。这可以通过布尔索引、切片或 `numpy.delete()` 函数来实现。

- 使用布尔索引

```
a = np.array([1, 2, 3, 4, 5])

# 删除值为3的元素
mask = a != 3
b = a[mask]
print(b) # 输出: [1 2 4 5]
```

- 切片可以用来跳过特定的元素或元素范围，但这种方法不如布尔索引灵活。

```
a = np.array([1, 2, 3, 4, 5])

# 跳过第三个元素
b = np.concatenate((a[:2], a[3:]))
print(b) # 输出: [1 2 4 5]
```

- `numpy.delete()` 函数可以从数组中删除指定位置的元素。

```
a = np.array([1, 2, 3, 4, 5])

# 删除索引为2的元素（即值为3的元素）
b = np.delete(a, 2)
print(b) # 输出: [1 2 4 5]
```

注意

- 这些操作都会返回一个新的数组，而不是修改原始数组。
- 对于大型数组，频繁地创建新数组（尤其是在循环中）可能会导致性能问题。在这些情况下，考虑是否可以使用更高效的算法或数据结构，或者是否可以将数据转换为更适合追加和删除操作的形式（如Python列表）。
- `numpy.append()` 和 `numpy.delete()` 虽然方便，但在处理大数据集时可能不是最高效的方法。在这些情况下，考虑使用布尔索引或切片等更底层的方法。

1.1.3 数组广播机制

广播机制是NumPy在算术运算中处理不同形状数组的一种方式，它允许在不进行显式复制数据的情况下执行元素级别的操作，从而提高了代码的执行效率和可读性。当两个或多个数组的形状不一致时，NumPy会尝试通过广播机制将它们“扩展”到相同的形状，以便可以进行逐元素的算术运算。

在NumPy中，当两个或多个数组的形状不一致，但满足广播规则时，就会发生数组广播。具体来说，如果两个数组在维度上不匹配，但满足以下条件之一，就可以通过广播机制进行运算：

1. **维度较小的数组会在前面补1**，使得两个数组的维度数相同。
2. **在某个维度上，如果一个数组的大小为1**，则它会被视为在该维度上重复扩展，以匹配另一个数组的大小。
3. **如果两个数组在所有维度上都不匹配，并且没有任何一个维度的大小为1**，则无法进行广播，并会引发错误。

案例一：简单的一维数组广播

```
import numpy as np

A = np.array([1, 2, 3]) # 形状 (3,)
B = np.array([10, 20, 30, 40]) # 形状 (4,)

# 尝试进行加法运算，但直接相加会报错，因为形状不匹配
# 但如果我们将A视为 (1, 3) 形状的数组，并将B视为 (4, 1) 形状的数组，然后进行逐元素加法，则可以实现广播
# 然而，NumPy不会直接显示地改变数组的形状，而是隐式地进行广播

# 实际上，NumPy不允许直接将形状为 (3,) 和 (4,) 的数组进行广播
# 但如果我们将B的前三个元素与A的元素相加，可以模拟广播的效果（注意：这不是NumPy的默认行为）
# 为了演示广播，我们可以使用形状兼容的数组
C = np.array([10, 20, 30]) # 形状 (3,)
D = A + C # 这里可以广播，因为形状相同
print(D) # 输出: [11 22 33]

# 注意：上面的C和D的示例是为了说明广播的概念，而不是NumPy直接允许的操作
```

注意：上面的示例中，直接使用A和B进行广播是无效的，因为它们的形状在所有维度上都不匹配，且没有任何一个维度的大小为1。但为了说明广播的概念，我使用了形状兼容的数组C来与A进行加法运算。

案例二：二维数组与一维数组的广播

```
import numpy as np

A = np.array([[1, 2, 3], [4, 5, 6]]) # 形状 (2, 3)
B = np.array([10, 20, 30]) # 形状 (3,)

# B会被视为形状为 (1, 3) 的数组，然后在第一个维度上复制以匹配A的形状 (2, 3)
# 现在A和B具有相同的“广播后”形状，可以进行逐元素加法
C = A + B
print(C)
# 输出:
# [[11 22 33]
#  [14 25 36]]
```

在这个例子中，B数组（形状为 (3,)）在第一个维度上被隐式地扩展为 (1, 3)，然后在这个维度上复制以匹配A数组的形状 (2, 3)。这样，A和B就可以进行逐元素的加法运算了。

通过这两个案例，我们可以看到NumPy的广播机制是如何工作的，以及它如何允许我们在不显式改变数组形状的情况下进行高效的数组运算。

1.1.4 随机数生成与抽样

NumPy是Python中用于科学计算的一个基础库，它提供了大量的数学函数工具，特别是针对数组的操作。在NumPy中，生成随机数数组是一个常见的需求，NumPy提供了多种函数来满足这一需求。以下是NumPy中几种常用的随机数数组生成方法及其使用方式：

1. numpy.random.rand()

功能：生成指定形状的数组，数组中的元素在区间[0, 1)内均匀分布。

使用示例：

```
import numpy as np

# 生成一个具有5个元素的一维数组
arr1 = np.random.rand(5)
print(arr1)

# 生成一个2行3列的二维数组
arr2 = np.random.rand(2, 3)
print(arr2)
```

2. numpy.random.randn()

功能：生成指定形状的数组，数组中的元素服从标准正态分布（均值为0，标准差为1）。

使用示例：

```
import numpy as np

# 生成一个具有5个元素的一维数组
arr1 = np.random.randn(5)
print(arr1)

# 生成一个2行3列的二维数组
arr2 = np.random.randn(2, 3)
print(arr2)
```

3. numpy.random.randint()

功能：生成指定范围内的随机整数数组。可以指定范围的最小值、最大值和数组的形状。

使用示例：

```
import numpy as np

# 生成一个在[1, 10)之间的一维整数数组
arr1 = np.random.randint(1, 10, size=5)
print(arr1)

# 生成一个在[1, 10)之间2行3列的二维整数数组
arr2 = np.random.randint(1, 10, size=(2, 3))
print(arr2)
```

4. numpy.random.uniform()

功能：生成指定范围内的均匀分布的随机数数组。需要指定生成随机数的下界、上界和数组的形状。

使用示例：

```
import numpy as np

# 生成一个在[1, 5)之间的一维数组
arr1 = np.random.uniform(1, 5, size=5)
print(arr1)

# 生成一个在[1, 5)之间2行3列的二维数组
arr2 = np.random.uniform(1, 5, size=(2, 3))
print(arr2)
```

5. numpy.random.normal()

功能：生成指定均值和标准差的正态分布的随机数数组。需要指定生成随机数的均值、标准差和数组的形状。

使用示例：

```
import numpy as np

# 生成均值为2，标准差为0.5的一维数组
arr1 = np.random.normal(2, 0.5, size=5)
print(arr1)

# 生成均值为2，标准差为0.5的2行3列的二维数组
arr2 = np.random.normal(2, 0.5, size=(2, 3))
print(arr2)
```

6. numpy.random.random()

功能：与 `numpy.random.rand()` 类似，生成指定形状的数组，数组中的元素在区间[0, 1)内均匀分布。该函数是Python标准库 `random` 模块的函数的向量化版本。

使用示例：

```
import numpy as np

# 生成一个3x3的二维数组
arr = np.random.random((3, 3))
print(arr)
```

7. numpy.random.seed()

功能：在生成伪随机数时指定种子。指定相同的种子将会产生相同的随机数序列，这在调试代码时非常有用。

使用示例：

```
import numpy as np

np.random.seed(0)
arr1 = np.random.rand(3, 3)
print(arr1) # 每次运行这段代码时，如果seed相同，生成的随机数序列也将相同
```

NumPy提供了丰富的随机数生成函数，可以满足各种生成随机数数组的需求。在实际应用中，可以根据具体需求选择合适的函数，并通过指定参数来生成满足需求的随机数数组。

1.2 Numpy数组的运算

1.2.1 向量的运算

1. 对元素的操作

如果要进行逐元素的运算就可能需要用到循环。和列表类似，我们可以利用for循环来遍历一个numpy.array。例如：

```
import numpy as np

# 创建一个Numpy数组
arr = np.array([1, 2, 3, 4, 5])

# 使用for循环遍历数组
for i in arr:
    print(i)
```

但是，如果在numpy当中你仍然大规模使用循环的话，它的效率提升不了一点。我们发明numpy的初衷就是希望能够以向量化并行计算的方式去提升运算效率。这里我们介绍几种利用向量批量操作的方法。

第一个我们在上一节当中提到。 `numpy.arange([start, ]stop, [step, ]dtype=None)` 函数类似于Python内置的 `range()` 函数，但返回的是一个数组，而不是一个迭代器或列表。它生成一个给定起始值 `start`（默认为0）、结束值 `stop`（不包括此值）和步长 `step`（默认为1）的数组。

```
# 使用numpy.arange
arr = np.arange(0, 10, 2) # 类似于range(0, 10, 2)
print(arr) # 输出: [0 2 4 6 8]
```

第二个方法

是 `numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)` 函数，用于生成一个等差数列，即在指定的间隔内返回均匀间隔的数字。与 `numpy.arange` 不同，`numpy.linspace` 是基于间隔数量（`num`）来生成数组的，而不是步长。

```
# 使用numpy.linspace
arr = np.linspace(0, 10, 5) # 在0到10之间生成5个等间距的点
print(arr) # 输出类似: [ 0.   2.5   5.   7.5 10.]
```

`numpy.arange`、`numpy.linspace`与`range`之间的用法存在一些互通之处，但也有不同点。它们都可以用来生成一系列的数字。不同点在于：

- **类型**： `range` 返回的是一个迭代器，而 `numpy.arange` 和 `numpy.linspace` 返回的是Numpy数组。
- **用途**： `range` 主要用于Python的for循环中迭代，而 `numpy.arange` 和 `numpy.linspace` 主要用于Numpy的数值计算中，特别是在需要数组操作时。
- **参数**： `numpy.arange` 和 `range` 基于起始值、结束值和步长生成序列，而 `numpy.linspace` 基于起始值、结束值和元素数量生成等差数列。
- **性能**： 在处理大型数据集时，Numpy数组（`numpy.arange` 和 `numpy.linspace` 生成的）比Python列表（`range` 转换为列表）或迭代（直接使用 `range`）在性能上有显著优势，因为Numpy数组在内存

中连续存储，并且支持向量化操作。

2. 向量的线性运算

好了，现在让我们切入今天的正题，就是如何使用np.array模拟一个向量。那么大家在中学阶段学的向量主要是二维欧式空间或三维欧式空间下的向量对吧，如果我告诉你一个向量的维度可以有更多维，大家还可以理解吗？哈哈，其实在numpy当中，一个一维数组就可以表示一个n维向量了。向量的概念其实是可以扩展到更高维空间当中的。

想象一下，我们手里拿着一堆乐高积木（其实就是Numpy数组啦，不过乐高更有趣点），现在我们要用这些积木来玩点“数学魔法”——向量的加减法和缩放运算。准备好了吗？魔法即将开始！

首先，我们有两个小伙伴，小明和小华，他们各自手里拿着一串乐高积木，代表两个向量。小明的向量是 $[1, 2, 3]$ ，而小华的是 $[4, 5, 6]$ 。他们想一起玩，所以决定把各自的积木加在一起。

在Numpy的世界里，这超级简单：

```
import numpy as np

# 小明的向量
ming_vector = np.array([1, 2, 3])
# 小华的向量
hua_vector = np.array([4, 5, 6])

# 向量加法
combined_vector = ming_vector + hua_vector

print("小明和小华的乐高积木加在一起变成了：", combined_vector)
# 输出：小明和小华的乐高积木加在一起变成了： [5 7 9]
```

看，就像把两堆乐高积木放在一起那么简单！

不过，有时候小朋友们也会闹别扭，比如小明觉得小华手上的乐高太多了，想要抢一点过来。这时候，我们就需要做减法了。

```
# 向量减法
difference_vector = hua_vector - ming_vector

print("小华从小明那里抢了一些乐高后，剩下的乐高是：", difference_vector)
# 输出：小华从小明那里抢了一些乐高后，剩下的乐高是： [3 3 3]
```

接下来，假设我们有一个神奇的放大镜，可以让我们手上的乐高积木变大或变小。这个放大镜就是“缩放运算”。

比如，我们用放大镜把小明手上的乐高积木都放大2倍：

```
# 缩放运算，放大2倍
scaled_ming_vector = ming_vector * 2

print("小明用放大镜把乐高积木放大2倍后，得到了：", scaled_ming_vector)
# 输出：小明用放大镜把乐高积木放大2倍后，得到了： [2 4 6]
```

通过这些简单的例子，我们可以看到，使用Numpy进行向量的加减法和缩放运算，就像玩乐高一样轻松愉快。在数学和计算机科学中，向量是基本且重要的概念，而Numpy库提供了强大的工具来处理这些操作。

对于任意两个维度相同的向量 $\vec{a} = (a_1, a_2, \dots, a_n)$ 和 $\vec{b} = (b_1, b_2, \dots, b_n)$ ，它们的加法定义为：

$$\vec{a} + \vec{b} = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$$

它们的减法定义为：

$$\vec{a} - \vec{b} = (a_1 - b_1, a_2 - b_2, \dots, a_n - b_n)$$

在Numpy中，这些操作可以直接通过 $+$ 和 $-$ 运算符完成。

向量的缩放运算涉及一个标量（即一个单一的数）和一个向量。给定一个标量 k 和一个向量 $\vec{a} = (a_1, a_2, \dots, a_n)$ ，向量的缩放定义为：

$$k\vec{a} = (ka_1, ka_2, \dots, ka_n)$$

在Numpy中，这可以通过将标量与向量相乘来实现。

对于更高维度的向量（例如四维或更高），Numpy的处理方式与上述相同。只需确保在进行运算时，涉及的向量维度相同即可。

```
# 定义两个四维向量
vec_c = np.array([1, 2, 3, 4])
vec_d = np.array([5, 6, 7, 8])

# 向量加法
vec_sum_4d = vec_c + vec_d
print("四维向量加法结果: ", vec_sum_4d) # 输出: 四维向量加法结果: [6 8 10 12]

# 向量缩放 (以标量3为例)
scaled_vec_4d = 3 * vec_c
print("四维向量缩放结果 (乘以3): ", scaled_vec_4d) # 输出: 四维向量缩放结果 (乘以3): [3 6 9 12]
```

3. 向量的非线性运算

加减法和数乘都属于是比较基础的线性运算，但向量还有一些非线性运算方法。**向量的数量积（又称点积、内积或标量积）**是一种特殊的向量运算，其结果是一个标量（即一个单一的数），而不是向量。向量的数量积定义为两个向量的大小、它们之间夹角的余弦值的乘积。具体地，对于两个向量 $\vec{a}$ 和 $\vec{b}$ ，它们的数量积表示为 $\vec{a} \cdot \vec{b}$ 或 $a \cdot b$ ，计算公式为：

$$\vec{a} \cdot \vec{b} = |\vec{a}| \times |\vec{b}| \times \cos \theta$$

其中， $|\vec{a}|$ 和 $|\vec{b}|$ 分别是向量 $\vec{a}$ 和 $\vec{b}$ 的模（即大小或长度）， θ 是向量 $\vec{a}$ 和 $\vec{b}$ 之间的夹角。

数量积具有一些重要的性质，例如：

- 当两个向量同向时，数量积为正；反向时，数量积为负；垂直时，数量积为零。
- 数量积满足交换律，即 $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$ 。
- 数量积的绝对值不超过两个向量模的乘积，即 $|\vec{a} \cdot \vec{b}| \leq |\vec{a}| \times |\vec{b}|$ 。

在Numpy中实现向量的数量积，可以使用 `numpy.dot()` 函数或 `@` 操作符。这两个操作在处理一维数组（可以视为向量）时，都会计算它们的点积（即数量积）。

使用 `numpy.dot()` 函数：


```
import numpy as np

# 定义两个向量
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 计算数量积
dot_product = np.dot(a, b)

print("数量积为: ", dot_product) # 输出: 数量积为: 32
```

使用@操作符 (Python 3.5+) :

```
import numpy as np

# 定义两个向量
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 计算数量积
dot_product = a @ b

print("数量积为: ", dot_product) # 输出: 数量积为: 32
```

在这两个示例中，a 和 b 是两个三维向量，我们通过Numpy的 dot() 函数或 @ 操作符计算了它们的数量积，并打印了结果。注意，虽然这两个操作符在处理一维数组时等同于计算数量积，但它们在处理多维数组时会有不同的行为（例如，@ 操作符在Python中用于矩阵乘法，而 numpy.dot() 在处理二维数组时也会进行矩阵乘法，但在处理更高维数组时行为较为复杂）。然而，在本例中，我们仅关注一维数组（向量）的情况。

这里可能有同学要问了，为什么这里不能直接用乘号呢？我们来看看这里如果改成乘号会变成什么东西：

```
array([4,10,18])
```

你会发现，结果变成了一个向量。这种计算叫逐元素乘法。那么如何把它变成正确的数量积呢？只需要对这个数组进行一趟求和就好了。

另一种非线性运算就是叉乘。向量的叉乘（Cross Product），也称为向量的叉积或向量积，是向量代数中的一个重要概念。它涉及到两个三维向量的运算，其结果是一个新的向量，这个新向量垂直于原来两个向量所构成的平面。

1. **定义**：对于两个三维向量 $\vec{a} = (a_1, a_2, a_3)$ 和 $\vec{b} = (b_1, b_2, b_3)$ ，它们的叉乘 $\vec{c} = \vec{a} \times \vec{b}$ 是一个新的向量，其方向垂直于 $\vec{a}$ 和 $\vec{b}$ 所在的平面，并且满足右手定则（即四指从 $\vec{a}$ 转向 $\vec{b}$ 时，大拇指的指向即为 $\vec{c}$ 的方向）。
2. **坐标表示**：在三维空间中，叉乘的坐标可以通过行列式的方式计算，具体为：

$$\vec{c} = \vec{a} \times \vec{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$$

其中， $\mathbf{i}$ 、 $\mathbf{j}$ 、 $\mathbf{k}$ 分别是沿 x 、 y 、 z 轴的单位向量。

3. 性质：

- 叉乘不满足交换律，即 $\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$ 。
- 叉乘的模等于两向量模的乘积与它们之间夹角的正弦值的积，即 $|\vec{a} \times \vec{b}| = |\vec{a}| \cdot |\vec{b}| \cdot \sin \theta$ 。
- 叉乘的结果垂直于原来两个向量所构成的平面。

在Numpy库中，可以直接使用 `numpy.cross()` 函数来计算两个三维向量的叉乘。这个函数接受两个一维数组（代表三维向量）作为输入，并返回一个一维数组（代表叉乘结果的三维向量）。

```
import numpy as np

# 定义两个三维向量
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 计算叉乘
c = np.cross(a, b)

# 输出结果
print("叉乘结果：", c) # 输出类似于：[ -3  6 -3]
```

在这个例子中，`a` 和 `b` 是两个三维向量，通过调用 `np.cross(a, b)` 计算它们的叉乘，并将结果存储在 `c` 中。最后，通过打印 `c` 来查看叉乘的结果。

注意：

- 叉乘是三维向量特有的运算，不适用于二维向量或更高维度的向量（尽管可以通过一些方式在二维空间中模拟叉乘的概念，但那不是真正的叉乘）。
- 在使用Numpy进行叉乘计算时，请确保输入的向量是一维数组，并且长度为3（代表三维空间中的坐标）。如果输入不符合这些要求，`numpy.cross()` 函数可能会引发错误。

在NumPy中，对数组进行排序、筛选等操作是非常常见的任务，特别是在处理科学计算和数据分析时。这里，我将详细介绍如何对NumPy数组进行排序、筛选，特别是针对多条件筛选和多维数组筛选的方法。

NumPy提供了 `numpy.sort()` 函数和数组的 `sort()` 方法对数组进行排序。`numpy.sort()` 函数返回排序后的数组副本，而数组的 `sort()` 方法则直接对数组本身进行原地排序。

```
import numpy as np

# 创建一个数组
arr = np.array([3, 1, 4, 1, 5, 9, 2])

# 使用numpy.sort()排序，返回新数组
sorted_arr = np.sort(arr)
print("Sorted array with numpy.sort:", sorted_arr)

# 使用数组的sort()方法排序，原地修改
arr.sort()
print("Array sorted in-place:", arr)
```

4. 向量的逻辑运算——筛选

筛选通常指的是基于某些条件选择数组中的元素。在NumPy中，这可以通过布尔索引、`numpy.where()` 函数、以及使用条件表达式来实现。

单条件筛选

```
# 假设我们有一个数组
arr = np.array([1, 2, 3, 4, 5, 6])

# 筛选所有大于3的元素
filtered_arr = arr[arr > 3]
print("Filtered array:", filtered_arr)
```

多条件筛选

对于多条件筛选，可以使用逻辑运算符（如 & 和 |）来组合多个条件。注意，在使用 & 和 | 时，需要确保每个条件都被包裹在括号中，并且可能需要使用 `numpy.logical_and()` 和 `numpy.logical_or()` 函数来确保逻辑操作是逐元素的。

```
# 创建一个二维数组
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 筛选所有行，其中第一个元素大于3且第二个元素小于8
# 注意使用&和括号
filtered_rows = arr_2d[(arr_2d[:, 0] > 3) & (arr_2d[:, 1] < 8)]
print("Filtered rows:", filtered_rows)

# 或者使用numpy.logical_and()
filtered_rows_logical = arr_2d[np.logical_and(arr_2d[:, 0] > 3, arr_2d[:, 1] < 8)]
print("Filtered rows with logical_and:", filtered_rows_logical)
```

1.2.2 矩阵的运算

1. 矩阵的算术运算

矩阵就像是数学世界里的超级大表格，每个小格子里都藏着一个数字，等着我们去探索。矩阵的加减法，就像是咱们在超市买完东西后，用计算器加加减减那些商品的价格一样，只不过这次咱们是在这些大表格上动手。

想象一下，你有两个一模一样的超级大表格，只不过每个格子里的数字可能不一样。矩阵加法呢，就是让你把这两个表格对应格子里的数字加在一起，然后得到一个新的表格。就像是两个朋友一起数钱，然后把各自的零钱合在一起，看看总共有多少。

在NumPy里，做这件事超级简单。你只需要用个加号 +，把两个矩阵（在NumPy里叫做数组）放在一起，NumPy就会自动帮你把对应位置的数字加起来，然后返回一个全新的数组。比如：

```
import numpy as np

# 创建两个矩阵（数组）
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# 矩阵加法
result_add = matrix1 + matrix2
print(result_add) # 输出: [[ 6  8] [10 12]]
```

看，简单到不行吧，就像是小学生做加法题一样。

矩阵的减法呢，就像是咱们在超市买完东西后，发现有些东西买多了，想要退掉一些。不过在这里，我们不是真的退东西，而是想象你有两个表格，然后从第一个表格的每个格子里减去第二个表格对应格子的数字。这就像是你和朋友一起数钱，但这次你决定要看看自己比朋友多了还是少了多少钱。

在NumPy里，做矩阵减法也是用个简单的减法符号 `-`，把两个矩阵放在一起，NumPy就会帮你把对应位置的数字相减，然后返回结果。比如：

```
# 接着上面的例子
result_subtract = matrix1 - matrix2
print(result_subtract) # 输出: [[-4 -4] [-4 -4]]
```

看，这就像是玩游戏里的“找不同”，只不过我们是找“差多少”。

矩阵的加减法在NumPy里就是这么简单直白，就像是我们平时做算术题一样。NumPy就像是那个超级智能的计算器，帮我们把那些复杂的数字运算变得轻而易举。所以，下次当你看到矩阵的加减法时，别害怕，想象成是在玩一个高级的“数独”游戏，只不过这次你是用加减号来填满格子，而不是数字。

矩阵的加减法**定义**：

设A和B是两个同型矩阵（即行数和列数分别相等），则矩阵A与B的和（或差）是一个与A、B同型的矩阵，记为 $A+B$ （或 $A-B$ ），其元素由A和B对应元素相加（或相减）得到。

运算法则与数学公式：

1. **加法交换律**： $A + B = B + A$ 。
 - 这意味着矩阵加法的顺序不影响最终结果。
2. **加法结合律**： $(A + B) + C = A + (B + C)$ 。

- 类似于数的加法，矩阵的加法也满足结合律。
3. **零矩阵与加法恒等元**： $A + O = A = O + A$ ，其中O是零矩阵（所有元素均为0的矩阵）。
 - 零矩阵在矩阵加法中起着类似于数0在数的加法中的作用，即任何矩阵与零矩阵相加都等于该矩阵本身。
 4. **负矩阵与减法**： $A - B = A + (-B)$ ，其中-B是B的负矩阵（即B中每个元素取反得到的矩阵）。
 - 矩阵的减法可以通过加法与负矩阵来实现。

矩阵的数乘**定义**：

设k是一个数，A是一个矩阵，则数k与矩阵A的数乘是一个新的矩阵，记为kA或Ak，其元素由A中每个元素乘以k得到。

运算法则与数学公式：

1. **结合律**： $(kl)A = k(lA)$ ，其中k和l是数，A是矩阵。
 - 这意味着数的乘法可以分配到矩阵的乘法中去。
2. **分配律**： $k(A + B) = kA + kB$ ，其中k是数，A和B是矩阵。
 - 类似于数的乘法分配律，数的乘法也可以分配到矩阵的加法中去。
3. **数乘恒等元**： $1A = A$ ，其中1是单位数（在实数中即为1）。
 - 单位数乘以任何矩阵都等于该矩阵本身。
4. **零数乘**： $0A = O$ ，其中0是零数（在实数中即为0），O是零矩阵。
 - 零数乘以任何矩阵都等于零矩阵。

矩阵的乘法，听起来像是数学里的超级英雄大战，但其实它更像是两个舞者在舞台上优雅地合作完成一套复杂的舞步。想象一下，你有两个矩阵，A和B，它们不是随便站着的，而是各自有着自己的“队形”（行和列）。矩阵的乘法，就是让A的“队员”（行）和B的“队员”（列）——配对，然后他们手拉手跳起舞来，最后形成一个新的队形——也就是结果矩阵C。

不过，这里有个小规矩：A的队员数量（行数）得和B的列数相匹配，这样他们才能手牵手。如果A有m行n列，B有n行p列，那么他们跳完舞后形成的新队形C就会有m行p列。

怎么跳这个舞呢？其实每个“舞步”都是一个数乘法和加法的过程。A的每一行都会和B的每一列进行“对话”，具体来说，就是A的某一行的每个元素都会和B的某一列的对应元素相乘，然后这些乘积加起来，就得到了C中对应位置的一个元素。

在NumPy里实现这个“舞蹈”超级简单，你只需要用一个星号*（但注意，这里的*在NumPy里对于矩阵来说其实是元素对元素的乘法，而不是矩阵乘法。矩阵乘法在NumPy里用@符号或者numpy.dot()函数）。不过，为了避免混淆，我们通常会使用@符号或者numpy.matmul()函数来进行矩阵乘法。

举个例子：

```

import numpy as np

# 创建两个矩阵A和B
# 假设A是2x3的矩阵，B是3x2的矩阵，这样它们就可以相乘了
A = np.array([[1, 2, 3],
               [4, 5, 6]])
B = np.array([[7, 8],
               [9, 10],
               [11, 12]])

# 使用@符号进行矩阵乘法
C = A @ B
# 或者使用numpy.matmul()函数，效果一样
# C = np.matmul(A, B)

print(C)
# 输出将会是：
# [[ 58  64]
#  [139 154]]
# 这就是A和B“跳舞”后的新队形C

```

看，就像变魔术一样，两个矩阵A和B通过矩阵乘法“跳”出了一个全新的矩阵C！在NumPy的帮助下，这个过程变得既简单又有趣，就像是在数学的乐园里玩了一场别开生面的“舞蹈游戏”。

刚刚的例子等于是以讲故事的方式告诉大家怎么进行矩阵乘法。矩阵的乘法是线性代数中的一个基本且重要的运算，它遵循特定的规则来生成一个新的矩阵。以下是矩阵乘法的定义：

定义：

设 A 是一个 $m \times n$ 矩阵，其元素为 a_{ij} （其中 $i = 1, 2, \dots, m$ 且 $j = 1, 2, \dots, n$ ）， B 是一个 $n \times p$ 矩阵，其元素为 b_{jk} （其中 $j = 1, 2, \dots, n$ 且 $k = 1, 2, \dots, p$ ）。则矩阵 A 与矩阵 B 的乘积 $C = AB$ 是一个 $m \times p$ 矩阵，其元素 c_{ik} 定义为：

$$c_{ik} = \sum_{j=1}^n a_{ij} b_{jk} \quad (i = 1, 2, \dots, m; k = 1, 2, \dots, p)$$

这里， $\sum_{j=1}^n$ 表示对 j 从 1 到 n 的所有值求和。换句话说， c_{ik} 是通过取 A 的第 i 行与 B 的第 k 列的对应元素相乘后的和得到的。也就是：

\$\$

$$C = AB = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$
$$\begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{bmatrix}$$

$$\begin{bmatrix} \sum_{j=1}^n a_{1j} b_{j1} & \sum_{j=1}^n a_{1j} b_{j2} & \cdots & \sum_{j=1}^n a_{1j} b_{jp} \\ \sum_{j=1}^n a_{2j} b_{j1} & \sum_{j=1}^n a_{2j} b_{j2} & \cdots & \sum_{j=1}^n a_{2j} b_{jp} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{j=1}^n a_{mj} b_{j1} & \sum_{j=1}^n a_{mj} b_{j2} & \cdots & \sum_{j=1}^n a_{mj} b_{jp} \end{bmatrix}$$

\$\$

注意：在公式中，矩阵 A 和 B 的乘积 C 的每个元素 c_{ik} 是通过内层求和（对 j 的求和）得到的，这反映了矩阵乘法的定义。此外，矩阵 C 的维度 $(m \times p)$ 由 A 的行数 m 和 B 的列数 p 决定，而 A 的列数 n 和 B 的行数 n 必须相等，以确保乘法运算的合法性。

2. 矩阵的逻辑运算——查找

当然了，在矩阵这种多维形式的数组下，除了执行算术运算，也可以进行增删改查这些操作。多维数组的筛选与一维数组类似，但你可以使用更复杂的条件，并且可以利用多维索引。在上面的例子中，我们

已经看到了如何使用 `[:, 0]` 和 `[:, 1]` 来索引二维数组的行和列。

```
# 假设我们有一个表示学生分数的二维数组
scores = np.array([[85, 92], [78, 85], [90, 95], [55, 60]])

# 筛选数学成绩（第二列）大于80的所有行
high_math_scores = scores[scores[:, 1] > 80]
print("Students with high math scores:", high_math_scores)
```

`np.where()` 函数是NumPy中非常有用的一个函数，它可以根据条件表达式返回满足条件的元素的索引或者根据条件从数组中选择元素。

`np.where(condition, [x, y])` 函数有三个参数，但通常我们只使用前两个参数：

- `condition`：条件表达式，返回一个布尔数组。
- `[x, y]`（可选）：如果提供了这两个参数，`np.where()` 将返回两个数组，第一个数组包含 `condition` 为True的元素的索引，第二个数组包含 `condition` 为False的元素的索引。如果只提供了 `condition` 参数，则 `np.where()` 仅返回满足条件的元素的索引。

如果不提供 `[x, y]` 参数，`np.where()` 返回的是一个元组，其中包含满足条件的元素的索引（以数组的形式）。

假设我们有一个数组，并且想要找到所有大于某个值的元素的索引，或者根据条件选择元素。

```
import numpy as np

# 创建一个数组
arr = np.array([1, 2, 3, 4, 5, 6])

# 使用 np.where() 找到所有大于3的元素的索引
indices = np.where(arr > 3)
print("Indices of elements greater than 3:", indices)

# 如果你想要基于条件获取实际的元素值
filtered_values = arr[arr > 3]
print("Values greater than 3:", filtered_values)

# 如果你想要同时获取True和False条件下的索引（尽管这在这个场景下可能不常见）
true_indices, false_indices = np.where(arr > 3, [True_indices, False_indices])
# 注意：上面的代码实际上不会按预期工作，因为np.where()不是设计来这样使用的。
# 正确的做法是仅使用condition参数，或者如果你有x和y数组，则使用它们来根据条件替换值。

# 一个更实际的例子，使用x和y来替换值
arr_replaced = np.where(arr > 3, 'High', 'Low')
print("Replaced array with 'High' and 'Low':", arr_replaced)
# 注意：这里arr_replaced的类型将是'object'，因为NumPy无法推断出所有元素都是字符串

# 对于多维数组
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# 找到所有第二列大于5的元素的行索引和列索引
rows, cols = np.where(arr_2d[:, 1] > 5)
print("Rows and cols where second column > 5:", rows, cols)
```

在上面的示例中，我们展示了如何使用 `np.where()` 来找到满足条件的元素的索引，以及如何使用它来根据条件替换数组中的值。对于多维数组，我们可以使用多维索引来指定条件。

注意，在多维数组的示例中，`np.where()` 返回了两个数组：第一个数组包含满足条件的元素的行索引，第二个数组包含列索引。这是因为 `np.where()` 总是返回一个元组，其元素数量与条件数组中的维度数量相匹配。

3. 矩阵上的统计

NumPy还提供了丰富的函数来计算数组的统计指标，包括平均值（`mean()`）、和（`sum()`）、累积和（`cumsum()`）、连乘积（`prod()`）、累计积（`cumprod()`）、中位数（`median()`）、标准差（`std()`）和方差（`var()`）等。这些函数不仅提高了数据处理的效率，还使得数据分析变得更加简单

和直观。下面通过一个具体的案例来展示NumPy中 `mean()` , `sum()` , `cumsum()` , `prod()` , `cumprod()` , `median()` , `std()` , 和 `var()` 这些统计指标计算方法的用法与输出结果。

假设我们有一个二维数组 `arr` , 表示某个班级三位同学的三门课成绩:

```
import numpy as np

# 创建一个二维数组, 表示三位同学的三门课成绩
arr = np.array([[85, 92, 88],
                [78, 90, 85],
                [90, 87, 92]])
```

1. `mean()`

计算所有成绩的平均值:

```
mean_all = np.mean(arr)
print("所有成绩的平均值:", mean_all) # 输出: 所有成绩的平均值: 87.5
```

计算每门课的平均成绩:

```
mean_each_subject = np.mean(arr, axis=0)
print("每门课的平均成绩:", mean_each_subject) # 输出: 每门课的平均成绩: [84.33333333 89.66666667]
```

2. `sum()`

计算所有成绩的总和:

```
sum_all = np.sum(arr)
print("所有成绩的总和:", sum_all) # 输出: 所有成绩的总和: 262
```

计算每门课的总成绩:

```
sum_each_subject = np.sum(arr, axis=0)
print("每门课的总成绩:", sum_each_subject) # 输出: 每门课的总成绩: [262 269 265] 注意这里应该是[262 269 265]
```

3. cumsum()

计算成绩的累积和（按行）：

```
cumsum_rows = np.cumsum(arr, axis=1)
print("按行的累积和:", cumsum_rows)
# 输出:
# [[ 85 177 265]
#   [ 78 168 253]
#   [ 90 177 269]]
```

4. prod()

计算所有成绩的连乘积：

```
prod_all = np.prod(arr)
print("所有成绩的连乘积:", prod_all) # 输出：所有成绩的连乘积：5217120
```

计算每位同学的总成绩连乘积（实际上不常用，但为展示用法）：

```
prod_each_student = np.prod(arr, axis=1)
print("每位同学的总成绩连乘积:", prod_each_student) # 输出：每位同学的总成绩连乘积：[73024 58140 685056]
```

5. cumprod()

计算成绩的累积积（按行）：

```
cumprod_rows = np.cumprod(arr, axis=1)
print("按行的累积积:", cumprod_rows)
# 输出:
# [[ 85 7225 633800]
#   [ 78 6240 530400]
#   [ 90 7830 685056]]
```

6. median()

计算所有成绩的中位数：

```
median_all = np.median(arr)
print("所有成绩的中位数:", median_all) # 输出: 所有成绩的中位数: 88.0
```

计算每门课的中位数成绩（注意，这里为了演示，我们将其转换为一维数组）：

```
median_each_subject = np.median(arr.ravel()) # ravel()将数组展平
print("每门课的中位数成绩:", median_each_subject) # 注意: 这实际上是所有成绩的中位数, 因为展平了
# 若要计算每门课的中位数, 应分别处理每列
median_each_subject_correct = np.median(arr, axis=0)
print("正确的每门课的中位数成绩:", median_each_subject_correct) # 输出:
```

4. 矩阵的结果保存

矩阵可能规模很大，那如何把来之不易计算得到的结果保存起来呢？我们看一个例子：

首先，我们进行两个数组的加法运算，然后将结果保存到一个文本文件中。

```
import numpy as np

# 创建两个数组
array1 = np.array([1, 2, 3, 4, 5])
array2 = np.array([5, 4, 3, 2, 1])

# 进行数组加法运算
result_array = array1 + array2

# 打印结果，查看加法运算后的数组
print("加法运算结果: ", result_array)

# 使用np.savetxt保存结果到文本文件
# 假设我们要将结果保存到名为'result.txt'的文件中
np.savetxt('result.txt', result_array, fmt='%d', delimiter=',', newline='\n')

# 注意: fmt参数指定了每个元素的格式, 这里是整数'%d'
# delimiter参数指定了元素之间的分隔符, 这里是逗号','
# newline参数指定了每行结束后的换行符, 这里是换行符'\n'
# 这些参数可以根据需要进行调整
```

在上述例子中，我们首先创建了两个一维数组 `array1` 和 `array2`，并进行了加法运算，得到了结果数组 `result_array`。然后，我们使用 `np.savetxt` 函数将 `result_array` 保存到名为 `result.txt` 的文本文件中。在 `np.savetxt` 函数中，我们通过 `fmt` 参数指定了每个元素的格式（这里是整数格式 `'%d'`），通过 `delimiter` 参数指定了元素之间的分隔符（这里是逗号，`,`），以及通过 `newline` 参数指定了每行结束后的换行符（这里是换行符 `\n`）。

`np.savetxt` 是一个非常灵活的函数，支持多种参数来定制输出文件的格式。除了上面提到的 `fmt`、`delimiter` 和 `newline` 参数外，还有一些其他常用的参数：

- `header`：用于在文件顶部添加注释或标题行。
- `footer`：用于在文件底部添加文本。
- `comments`：指定注释字符，默认为 `'# '`。如果 `header` 或 `footer` 参数中包含该字符，它们将被视为注释。

例如，我们可以向文件添加标题和注释：

```
np.savetxt('result_with_header.txt', result_array, fmt='%d', delimiter=',',
           header='这是加法运算的结果', comments='# ')
```

这样，在 `result_with_header.txt` 文件的顶部将会有有一个注释行 `# 这是加法运算的结果`。

如果想要打开，只需要使用 `loadtxt` 进行开启即可：

```
# 使用loadtxt加载之前保存的数组
# 根据保存时指定的格式和分隔符来加载
loaded_arr = np.loadtxt('array_data.txt', delimiter=',', dtype=float)

# 如果你保存时没有指定fmt或者delimiter，这里也需要相应地调整
# 比如，如果默认分隔符是空格，且没有指定fmt
# loaded_arr = np.loadtxt('array_data_no_fmt.txt', dtype=float)

# 打印加载的数组来验证
print(loaded_arr)
```

通过调整这些参数，我们可以轻松地根据需保存数组数据到文本文件中，便于后续的分析或共享。

1.3 Numpy与线性代数

1.3.1 基本线性代数运算

1. 逆矩阵

首先我们来看看矩阵的求逆。

逆矩阵是线性代数中的一个基本概念，主要用于解线性方程组、矩阵的分解以及计算机图形学等领域。对于一个给定的方阵（即行数和列数相等的矩阵） A ，如果存在另一个方阵 B ，使得 $AB = BA = I$ （其中 I 是单位矩阵，即对角线上元素为1，其余元素为0的矩阵），则称 B 为 A 的逆矩阵，记作 A^{-1} 。

逆矩阵的求法可以用很多方法求，其中比较典型的包括：

1. 使用高斯-约旦消元法 (Gauss-Jordan Elimination)：

高斯-约旦消元法是一种将矩阵通过行变换化为单位矩阵的算法，其过程中可以同时求得原矩阵的逆矩阵（如果存在的话）。这种方法的基本思想是通过行变换将矩阵转换为单位矩阵，同时记录下这些行变换，这些变换的逆变换即为原矩阵的逆矩阵。

2. 使用伴随矩阵与行列式：

如果矩阵 A 的行列式 $|A| \neq 0$ （即矩阵是可逆的），则 A 的逆矩阵可以通过其伴随矩阵 $adj(A)$ 与行列式 $|A|$ 的倒数来计算： $A^{-1} = \frac{1}{|A|} adj(A)$ 。伴随矩阵 $adj(A)$ 是矩阵 A 的代数余子式矩阵的转置。

在Python中，NumPy库提供了非常方便的函数来计算矩阵的逆。如果你已经安装了NumPy，可以使用 `numpy.linalg.inv` 函数来计算逆矩阵。以下是一个简单的示例：

```
import numpy as np

# 定义一个方阵A
A = np.array([[1, 2], [3, 4]])

# 计算A的逆矩阵
A_inv = np.linalg.inv(A)

print("矩阵A:")
print(A)
print("矩阵A的逆矩阵:")
print(A_inv)

# 验证A * A_inv 是否等于单位矩阵
I = np.dot(A, A_inv)
print("A * A_inv =")
print(I)
```

注意，如果矩阵 A 不可逆（即其行列式为0），`np.linalg.inv(A)`将抛出一个 `LinAlgError` 错误。因此，在实际应用中，你可能需要先检查矩阵是否可逆，例如通过计算其行列式：

```
if np.linalg.det(A) != 0:
    A_inv = np.linalg.inv(A)
else:
    print("矩阵A不可逆")
```

而伪逆矩阵的出现，主要是为了解决在线性代数中遇到的一些问题，特别是在处理非方阵（即行数和列数不相等的矩阵）或奇异矩阵（即行列式为0的方阵）时，这些矩阵没有传统意义上的逆矩阵。伪逆矩阵提供了一种在这些情况下求解线性方程组或最小化二乘误差的方法。

在NumPy库中，`np.linalg.pinv` 函数用于计算矩阵的伪逆。这个函数可以处理非方阵和奇异矩阵，并返回它们的伪逆矩阵。


```
import numpy as np

# 定义一个非方阵或奇异方阵A
A = np.array([[1, 2, 3], [4, 5, 6]]) # 非方阵示例

# 计算A的伪逆矩阵
A_pinv = np.linalg.pinv(A)

print("矩阵A:")
print(A)
print("矩阵A的伪逆矩阵:")
print(A_pinv)

# 如果A是方阵且非奇异，A_pinv将接近于A的逆矩阵（但不是完全相同，因为计算方法和数值稳定性差异）
# 如果A是非方阵或奇异方阵，A_pinv将提供一个在某种意义上“最优”的逆矩阵
```

注意事项：

- 伪逆矩阵不是唯一的，但 `np.linalg.pinv` 函数通常会返回一个满足特定条件的伪逆矩阵（如最小化二范数解）。
- 当矩阵A是方阵且非奇异时，`np.linalg.pinv(A)` 的结果将接近于但通常不完全等于 `np.linalg.inv(A)`，因为计算伪逆的算法和计算逆矩阵的算法在数值上可能存在微小差异。
- 在使用伪逆矩阵时，应注意其应用场景和限制条件，以确保得到的结果符合实际需求。

2. 矩阵的秩

矩阵的秩是线性代数中的一个重要概念，它表示矩阵中行或列之间的线性关系。具体来说，矩阵的秩定义为该矩阵所有子式中非零子式的最高阶数。换句话说，如果我们从矩阵中挑选出一些行和列，组成一个子式，那么这个子式的阶数最大能是多少，这个数就是矩阵的秩。在另一种等价定义中，矩阵的秩也可以理解为矩阵中独立行（或列）的数量，即矩阵的行向量组（或列向量组）的秩。

矩阵的秩有很多重要的性质和应用，比如它可以用于判断矩阵是否可逆（非零方阵的秩等于其阶数时，矩阵可逆），以及确定线性方程组解的存在性和唯一性（如果系数矩阵的秩小于增广矩阵的秩，则方程组无解；如果系数矩阵的秩等于方程个数，则方程组有唯一解；如果系数矩阵的秩小于方程个数，则方程组有无穷多解）。

在Python中，NumPy库提供了非常方便的函数来计算矩阵的秩。具体来说，可以使用 `numpy.linalg.matrix_rank` 函数来计算矩阵的秩。以下是一个使用NumPy求矩阵秩的示例：

```
import numpy as np

# 定义一个矩阵
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 计算矩阵的秩
rank = np.linalg.matrix_rank(matrix)

print("矩阵的秩为:", rank)
```

在这个示例中，我们首先导入了NumPy库，并定义了一个3x3的矩阵。然后，我们使用 `np.linalg.matrix_rank` 函数来计算这个矩阵的秩，并将结果打印出来。注意，虽然这个矩阵的所有元素都是连续的整数，但根据矩阵秩的定义，它的秩并不等于3，而是小于3（在这个具体例子中，秩为2，因为矩阵的第三行是前两行的线性组合）。

3. 行列式

行列式是一个数学概念，用于描述一个矩阵的性质。具体来说，行列式是一个方阵中所有元素按照一定规则排列后所得到的一个标量值。它反映了矩阵的某种固有属性，如矩阵是否可逆、是否存在非零向量被映射到零向量等。

行列式的计算公式对于不同大小的矩阵有所不同。以下是一些常见的公式：

- 对于2x2矩阵A，其行列式|A|的公式为：

$$|A| = a_{11}a_{22} - a_{12}a_{21}$$

其中， a_{ij} 表示矩阵A中第i行第j列的元素。

- 对于3x3矩阵A，其行列式|A|的公式更为复杂，通常使用拉普拉斯展开或代数余子式的方法来计算。一个简化的公式（按第一行展开）为：

$$|A| = a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})$$

对于更大的矩阵，行列式的计算通常需要使用递归方法或特定的数学软件来辅助。

在Python中，NumPy库提供了 `numpy.linalg.det()` 函数来计算矩阵的行列式。以下是一个具体的示例：

```
import numpy as np

# 创建一个3x3的矩阵
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 0]])

# 计算矩阵的行列式
det = np.linalg.det(matrix)

# 输出结果
print("矩阵:\n", matrix)
print("行列式:", det)
```

在这个示例中，我们首先创建了一个3x3的矩阵 `matrix`，然后使用 `np.linalg.det(matrix)` 计算了它的行列式，并将结果存储在变量 `det` 中。最后，我们输出了矩阵和它的行列式值。需要注意的是，由于浮点数的精度问题，计算得到的行列式值可能会略有不同。如果需要高精度的计算，可以考虑使用其他数学软件或库。

4. 范数

矩阵范数用于衡量矩阵的“大小”或“规模”。它是一个将矩阵映射到非负实数的函数，满足一系列特定的性质，如非负性、齐次性、三角不等式等。矩阵范数有多种定义方式，常见的有L1范数、L2范数（也称为弗罗贝尼乌斯范数）、无穷范数等。

以弗罗贝尼乌斯范数为例，对于矩阵 $A \in \mathbb{R}^{m \times n}$ ，其弗罗贝尼乌斯范数定义为：

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

其中， a_{ij} 是矩阵A中第i行第j列的元素。这个范数实际上是矩阵元素平方和的平方根，类似于向量的欧几里得范数。

范数和行列式的异同点

相同点：

- 它们都是矩阵的某种“度量”，用于描述矩阵的某种性质或大小。

不同点：

- **定义和性质**：行列式是一个标量值，与矩阵的行列变换和逆矩阵等性质紧密相关；而矩阵范数是一个将矩阵映射到非负实数的函数，满足非负性、齐次性和三角不等式等性质。
- **用途**：行列式主要用于判断矩阵是否可逆、求解线性方程组、计算矩阵的特征多项式等；而矩阵范数则更多地用于矩阵的数值分析、稳定性分析、条件数计算等领域。
- **计算复杂度**：一般来说，计算行列式的复杂度相对较低（特别是对于小型矩阵或特殊类型的矩阵）；而计算矩阵范数的复杂度可能较高，特别是对于大型矩阵或需要高精度计算的场景。

在NumPy中，可以使用 `numpy.linalg.norm()` 函数来计算矩阵的范数。这个函数非常灵活，可以通过设置不同的参数来计算不同类型的范数。

以下是一个使用NumPy计算矩阵弗罗贝尼乌斯范数的示例：

```
import numpy as np

# 定义一个矩阵
A = np.array([[1, 2], [3, 4]])

# 计算矩阵的弗罗贝尼乌斯范数（默认参数即为弗罗贝尼乌斯范数）
norm_A = np.linalg.norm(A)

# 或者显式指定范数为'fro'
norm_A_fro = np.linalg.norm(A, 'fro')

# 输出结果
print("矩阵A的弗罗贝尼乌斯范数为:", norm_A)
print("矩阵A的弗罗贝尼乌斯范数（显式指定）为:", norm_A_fro)

# 如果想要计算其他类型的范数（如L1范数或无穷范数），可以修改norm函数的参数
norm_A_1 = np.linalg.norm(A, 1) # 计算L1范数（列和范数）
norm_A_inf = np.linalg.norm(A, np.inf) # 计算无穷范数（行和范数）

print("矩阵A的L1范数为:", norm_A_1)
print("矩阵A的无穷范数为:", norm_A_inf)
```

在这个例子中，我们首先定义了一个2x2的矩阵 A，然后使用 `numpy.linalg.norm()` 函数计算了它的弗罗贝尼乌斯范数、L1范数和无穷范数，并将结果打印出来。注意，`numpy.linalg.norm()` 函数的第一个参数是待计算范数的矩阵，第二个参数是范数的类型（默认为弗罗贝尼乌斯范数）。

1.3.2 线性方程组的求解

1. 利用克拉默法则

克拉默法则是线性代数中一个用于求解线性方程组（方程组中方程的个数与未知数的个数相等，且系数矩阵的行列式不为零）的定理。它表述为：如果线性方程组

$$Ax = b$$

的系数矩阵 A 是一个 $n \times n$ 矩阵，且 A 的行列式 $|A| \neq 0$ ，则方程组有唯一解，且解的第 i 个分量 x_i 可以用行列式表示为：

$$x_i = \frac{\det(A_i)}{\det(A)}$$

其中， A_i 是将 A 的第 i 列替换为向量 b 后得到的矩阵。

由于NumPy库提供了计算行列式的 `numpy.linalg.det()` 函数，我们可以利用这个函数来实现克拉默法则。

```

import numpy as np

def cramer_rule(A, b):
    """
    使用克拉默法则求解线性方程组  $Ax = b$ 
    :param A: 系数矩阵，形状为 (n, n)
    :param b: 常数项向量，形状为 (n,)
    :return: 方程组的解，形状为 (n,)
    """
    n = A.shape[0] # 方程组的未知数个数
    if A.shape[1] != n or b.shape[0] != n:
        raise ValueError("系数矩阵A和常数项向量b的维度不匹配")

    det_A = np.linalg.det(A) # 计算系数矩阵A的行列式
    if det_A == 0:
        raise ValueError("系数矩阵A的行列式为零，方程组无解或有无穷多解")

    x = np.zeros(n) # 初始化解向量
    for i in range(n):
        # 构造矩阵Ai，即将A的第i列替换为b
        Ai = A.copy()
        Ai[:, i] = b
        # 计算Ai的行列式并除以A的行列式得到解的第i个分量
        x[i] = np.linalg.det(Ai) / det_A

    return x

# 示例
A = np.array([[4, 3, 2], [2, -2, 4], [3, 8, 2]])
b = np.array([1, 5, 6])

solution = cramer_rule(A, b)
print("方程组的解为:", solution)

```

在这个程序中，我们首先检查系数矩阵 A 和常数项向量 b 的维度是否匹配，然后计算 A 的行列式。如果行列式为零，则抛出异常，因为此时方程组可能无解或有无穷多解。接下来，我们遍历每个未知数，构造相应的 A_i 矩阵，并计算其行列式，最后通过克拉默法则的公式求解出每个未知数的值。最后，返回解向量 x 。

2. 高斯消元法

高斯消元法是一种用于解线性方程组的算法，它通过对方程组的系数矩阵进行行变换（包括乘以非零常数、行交换和行相加），将其转化为上三角矩阵形式，进而简化求解过程。当系数矩阵转化为上三角矩阵后，可以逐行求解出每个未知数的值。

高斯消元法通常包括两个主要阶段：前向消元（也称为部分选主元高斯消元法）和回代过程。前向消元用于将矩阵转化为上三角形式，而回代过程则用于从上到下求解每个未知数的值。

我们可以手动实现高斯消元法以更好地理解其工作原理。以下是一个使用NumPy实现高斯消元法的简单程序：

```

import numpy as np

def gaussian_elimination(A, b):
    """
    使用高斯消元法求解线性方程组  $Ax = b$ 
    :param A: 系数矩阵，形状为 (n, n)
    :param b: 常数项向量，形状为 (n,)
    :return: 方程组的解，形状为 (n,)
    """
    n = A.shape[0]
    if A.shape[1] != n or b.shape[0] != n:
        raise ValueError("系数矩阵A和常数项向量b的维度不匹配")

    # 合并A和b以简化操作
    Ab = np.hstack((A, b.reshape(-1, 1)))

    # 前向消元过程
    for k in range(n):
        # 选择主元（这里简单起见，不进行部分选主元，实际应用中可能需要）
        # 如果当前行的主元为0，则应该进行行交换（这里省略了行交换的逻辑）

        # 将当前行以下的所有行减去当前行的适当倍数，使得以下行的第k列变为0
        for i in range(k+1, n):
            factor = Ab[i, k] / Ab[k, k]
            Ab[i, k:] -= factor * Ab[k, k:]

    # 回代过程
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = (Ab[i, -1] - np.dot(Ab[i, :i], x[:i])) / Ab[i, i]

    return x

# 示例
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]])
b = np.array([8, -11, -3])

solution = gaussian_elimination(A, b)
print("方程组的解为:", solution)

```

在这个程序中，我们首先检查系数矩阵 A 和常数项向量 b 的维度是否匹配。然后，我们创建一个新的矩阵 Ab ，将 A 和 b 合并成一个增广矩阵，以便在消元过程中同时处理系数和常数项。接下来，我们进行

前向消元过程，将矩阵转化为上三角形式。最后，我们通过回代过程从上到下求解出每个未知数的值，并返回解向量 x 。

需要注意的是，这个实现为了简化起见，没有包含部分选主元的逻辑，也没有处理主元为0（即需要行交换）的情况。在实际应用中，这些情况都需要妥善处理以确保算法的鲁棒性。此外，对于大型稀疏矩阵，可能还需要使用更高效的稀疏矩阵存储和运算技术来优化算法性能。

3. `numpy.linalg.solve` 函数

`numpy.linalg.solve` 函数用于求解线性方程组或矩阵方程。给定一个方阵 a 和一个一维数组 b ，`solve` 函数会找到 x ，使得 $ax = b$ 。这里， a 的维度是 (n, n) ， b 的维度是 $(n,)$ ，解 x 的维度也将是 $(n,)$ 。

用法：

```
numpy.linalg.solve(a, b)
```

- **参数：**

- a ：系数矩阵，一个 (n, n) 的二维数组或矩阵。
- b ：常数项向量，一个 $(n,)$ 形状的一维数组或矩阵。

- **返回值：**

- 返回一个一维数组，表示方程组的解。

注意：

- a 必须是方阵且可逆（即行列式不为零）。
- 如果 a 不是方阵或不可逆，函数将抛出 `LinAlgError` 异常。

`numpy.linalg.lstsq` 函数用于求解最小二乘问题。当方程的数量多于未知数的数量（即，超定系统）时，`lstsq` 可以找到使残差的平方和最小的解。此外，它也可以用于求解恰好确定或欠定的系统。

目的：

- 求解形如 $Ax = b$ 的线性方程组的最小二乘解，其中 A 的列可能多于行数，即存在多于一个可能的解，但我们需要找到使残差平方和最小的那个解。

用法：

```
numpy.linalg.lstsq(a, b, rcond=None)[0]
```

- **参数：**

- `a`：系数矩阵，一个 (m, n) 的二维数组或矩阵，其中 m 是方程的数量， n 是未知数的数量。
- `b`：常数项向量或矩阵，形状为 $(m,)$ 或 (m, k) ，其中 k 是方程组的数量（如果同时求解多个方程组）。
- `rcond`：用于确定有效秩的截止条件。如果 `rcond` 为 `None`，则使用机器精度的估计值。

- **返回值：**

- 返回一个元组，其中包含最小二乘解、残差、秩、奇异值和条件数。通常，我们只对第一个元素（最小二乘解）感兴趣。

注意：

- `lstsq` 函数返回的是所有可能解中残差平方和最小的那一个。
- 如果 `A` 是方阵且可逆，则 `lstsq` 的解与 `solve` 的解相同（在数值误差范围内）。
- 对于欠定系统（未知数多于方程），`lstsq` 会找到使残差平方和最小的解之一，这通常不是唯一的。

示例：

```
import numpy as np

# 系数矩阵A
A = np.array([[1, 2], [3, 4], [5, 6]])
# 常数项向量b
b = np.array([1, 2, 3])

# 使用lstsq求解最小二乘解
x, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
print("最小二乘解:", x)
```

在这个例子中，由于 `A` 的行数多于列数，因此方程组是超定的，`lstsq` 会找到使残差平方和最小的解。

1.3.3 矩阵分解

1. 特征值分解

矩阵的特征值分解（Eigenvalue Decomposition, EVD）是一种将矩阵分解为其特征值和特征向量表示的矩阵之积的方法，也称为谱分解（Spectral Decomposition）。这种方法在数值计算、机器学习、图像处理、计算机视觉等领域有着广泛的应用。

设A是一个n×n的矩阵，λ是一个数，若存在n维非零向量x，使得 $Ax = \lambda x$ ，则称λ是矩阵A的特征值，x是A对应于λ的特征向量。矩阵A的特征值分解就是将A分解为以下形式：

$$A = U\Lambda U^{-1}$$

或者，在U为正交矩阵（即U的列向量是单位向量且两两正交）的情况下，由于正交矩阵的逆等于其转置，上式可以写为：

$$A = U\Lambda U^T$$

其中，

- U是由A的特征向量组成的n×n矩阵，其列向量是A的单位化特征向量。
- Λ是一个n×n的对角矩阵，其对角线上的元素是A的特征值，非对角线上的元素都是0。

注意事项

1. **特征值与特征向量的关系**：特征值是矩阵中与特定特征向量相关的实数，它表示该特征向量在矩阵变换下的缩放比例。特征向量是矩阵变换中方向不变的向量。
2. **矩阵的可对角化**：只有可对角化矩阵（即存在足够数量的线性无关特征向量的矩阵）才能进行特征值分解。
3. **应用领域**：特征值分解在信号处理、统计学、经济学、量子力学等多个领域都有重要应用。例如，在信号处理中，特征值可以用来分析信号的频率成分；在统计学中，主成分分析（PCA）就是基于特征值分解的一种数据降维方法。

在NumPy中，进行特征值分解主要使用 `numpy.linalg.eig` 函数。这个函数计算给定方阵的特征值和右特征向量。对于对称矩阵（或更一般地，Hermitian矩阵），通常还推荐使用 `numpy.linalg.eigh`，因为它在数值上更稳定且效率更高。不过，对于非对称矩阵，应使用 `numpy.linalg.eig`。

以下是如何使用 `numpy.linalg.eig` 进行特征值分解的示例：

```

import numpy as np

# 定义一个方阵A
A = np.array([[4, 2], [1, 3]])

# 使用eig函数进行特征值分解
eigenvalues, eigenvectors = np.linalg.eig(A)

# eigenvalues是一个包含特征值的数组
# eigenvectors的列是对应的特征向量

print("特征值: ", eigenvalues)
print("特征向量: ")
for i, eigenvector in enumerate(eigenvectors.T):
    print(f"特征值 {eigenvalues[i]} 对应的特征向量: {eigenvector}")

# 验证特征值和特征向量的正确性
# 对于每个特征值lambda_i和对应的特征向量v_i, 应有 Av_i = lambda_i * v_i
for i, eigenvalue in enumerate(eigenvalues):
    eigenvector = eigenvectors[:, i]
    result = np.dot(A, eigenvector)
    assert np.allclose(result, eigenvalue * eigenvector)
    print(f"验证特征值 {eigenvalue} 和特征向量 {eigenvector} 正确")

```

接下来我们可以看到两个用了矩阵分解的案例：

案例1：层次分析法

层次分析法是美国运筹学家匹茨堡大学教授萨蒂于20世纪70年代初提出的一种评价策略。这种策略虽然带有一定主观性，但非常奏效，也是在社会科学研究中经常使用的一类方法。

首先，层次分析法的流程分五步走：

1. 选择指标，构建层次模型。
2. 对目标层到准则层之间和准则层到方案层之间构建比较矩阵。
3. 对每个比较矩阵计算CR值检验是否通过CR检验，如果没有通过检验需要调整比较矩阵。
4. 求出每个矩阵最大的特征值对应的归一化权重向量。
5. 根据不同矩阵的归一化权向量计算出不同方案的得分进行比较。

根据上述流程，我们可以亲手写出一个AHP方法的代码：

```

def AHP(A):
    m=len(A)                                #获取指标个数
    n=len(A[0])
    RI=[0, 0, 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45, 1.49, 1.51]
    R= np.linalg.matrix_rank(A)             #求判断矩阵的秩
    V,D=np.linalg.eig(A)                    #求判断矩阵的特征值和特征向量，V特征值，D特征向量；
    list1 = list(V)
    B= np.max(list1)                         #最大特征值
    index = list1.index(B)
    C = D[:, index]                         #对应特征向量
    CI=(B-n)/(n-1)                          #计算一致性检验指标CI
    CR=CI/RI[n]
    if CR<0.10:
        print("CI=", CI.real)
        print("CR=", CR.real)
        print('对比矩阵A通过一致性检验，各向量权重向量Q为： ')
        sum=np.sum(C)
        Q=C/sum                             #特征向量标准化
        print(Q.real)                       # 输出权重向量
        return Q.real
    else:
        print("对比矩阵A未通过一致性检验，需对对比矩阵A重新构造")
        return 0

```

案例2：主成分分析

主成分分析的主要目的是希望用较少的变量去解释原来资料中的大部分变异，将原始数据中许多相关性较高的变量转化成彼此相互独立或不相关的变量。通常是选出比原始变量个数少，能解释大部分资料中的变异的几个新变量（也就是主成分）。因此，我们可以知道主成分分析的一般目的是：(1)数据的降维；(2)主成分的解释。

主成分分析包括以下几个步骤.

(1) 数据的去中心化：对数据表 X 中每个属性减去这一列的均值。这样做的目的在于消除数据平均水平对它的影响。

$$\bar{x} = (x_1 - \bar{x}_1, x_2 - \bar{x}_2, \dots, x_n - \bar{x}_n)$$

(2) 求协方差矩阵：注意这里需要除(n-1)。

$$C = \frac{1}{n-1} \bar{X}^T \bar{X}$$

(3) 对协方差矩阵进行特征值分解。

$$C = Q \Sigma Q^T$$

(4) 特征值排序：挑选更大的 k 个特征值，将特征向量组成矩阵 P 。

(5) 进行线性变换 $F = PX$ ，从而得到主成分分析后的矩阵。矩阵中的每一列称作一个主成分，选择的特征值经过归一化就变成了权重。

注意：实际过程中PCA的底层做矩阵分解时使用奇异值分解（SVD）更多一些，这里用特征值分解更加容易理解。

我们同样可以写出它的实现代码：

```
def pca(X,n_components):
    X=np.array(X)
    X=X-np.mean(X)
    n=len(X)
    A=np.dot(X.T,X)/(n-1)
    V,D=np.linalg.eig(A)
    idx = (-V).argsort(axis=None)[:n_components]
    P=D[idx]
    F=np.dot(X,P.T)
    return V[idx]/sum(V),F
```

2. 奇异值分解

奇异值分解（Singular Value Decomposition，简称SVD）是线性代数中一种重要的矩阵分解方法，它在多个领域如信号处理、统计学等中都有重要应用。以下是奇异值分解的具体定义：

奇异值分解是指将一个非零的 $m \times n$ 实矩阵 A （其中 $A \in R^{(m \times n)}$ ），表示为以下三个实矩阵乘积形式的运算，即进行矩阵的因子分解：

$$A = U \Sigma V^T$$

其中：

- U 是 $m \times m$ 阶正交矩阵 (orthogonal matrix) , 它的列向量称为左奇异向量。
- Σ 是 $m \times n$ 阶对角矩阵, 其对角线上的元素 (非负实数) 按降序排列, 称为奇异值。除了对角线上的元素外, Σ 的其他位置元素均为0。
- V 是 $n \times n$ 阶正交矩阵, 它的列向量称为右奇异向量。
- V^T 是 V 的转置矩阵。

它具备如下一些性质

1. **存在性**: 任意 $m \times n$ 的矩阵 A 的奇异值分解一定存在, 但可能不唯一 (特别是当矩阵 A 存在退化的奇异值时) 。
2. **最优近似**: 奇异值分解可以被视为在平方损失 (Frobenius范数) 意义下对矩阵的最优近似。在实际应用中, 常常通过截断奇异值分解 (只取最大的 k 个奇异值) 来实现矩阵的近似表示, 这在数据压缩等领域尤为重要。
3. **几何意义**: 从线性变换的角度来看, $m \times n$ 矩阵 A 表示从 n 维空间到 m 维空间的一个线性变换。奇异值分解可以将这个线性变换分解为三个简单的变换: 一个坐标系的旋转或反射变换、一个坐标轴的缩放变换 (缩放因子即为奇异值)、另一个坐标系的旋转或反射变换。

奇异值分解在多个领域有广泛应用, 包括但不限于:

- **数据压缩**: 通过截断奇异值分解, 可以实现对原始数据的近似表示, 从而达到数据压缩的目的。
- **信号处理**: 在信号处理中, 奇异值分解被用于分析信号的频率成分和进行信号重建。
- **统计学**: 在统计学中, 奇异值分解被用于主成分分析 (PCA), 一种数据降维方法, 用于找出数据中的主要模式。

在NumPy中, 进行奇异值分解 (SVD) 主要使用 `numpy.linalg.svd` 函数。这个函数返回给定矩阵的奇异值分解结果, 包括奇异值、左奇异向量和右奇异向量。

以下是如何使用 `numpy.linalg.svd` 进行奇异值分解的示例:

```

import numpy as np

# 定义一个矩阵A
A = np.array([[1, 2], [3, 4], [5, 6]])

# 使用svd函数进行奇异值分解
U, s, VT = np.linalg.svd(A, full_matrices=False)

# 注意: numpy.linalg.svd返回的s是奇异值数组, 而不是对角矩阵
# 我们需要将其转换为对角矩阵形式
Sigma = np.zeros((A.shape[0], A.shape[1]))
Sigma[:min(A.shape[0], A.shape[1]), :min(A.shape[0], A.shape[1])] = np.diag(s)

# VT是V的转置, 如果需要V, 则可以使用VT.T获取
V = VT.T

# 验证SVD的正确性
# A应该等于U * Sigma * VT (注意Sigma不是方阵时, 需要按照形状进行乘法)
reconstructed_A = np.dot(U, np.dot(Sigma, VT))

# 由于浮点运算的精度问题, 我们不会直接比较A和reconstructed_A是否相等
# 而是检查它们是否足够接近
print(np.allclose(A, reconstructed_A)) # 应该输出True或非常接近True的值

# 输出奇异值、左奇异向量和右奇异向量
print("奇异值: ", s)
print("左奇异向量: ")
print(U)
print("右奇异向量: ")
print(V)

```

请注意几个关键点:

1. `np.linalg.svd` 的 `full_matrices` 参数控制返回的矩阵U和V的形状。当 `full_matrices=True` 时（默认值），U和V都是方阵，且大小分别为 $m \times m$ 和 $n \times n$ （其中m和n是A的行数和列数）。然而，在许多情况下，我们不需要完整的U和V矩阵，只需要包含必要信息的“经济”版本。这可以通过设置 `full_matrices=False` 来实现，此时U的形状为 $m \times k$ ，V的形状为 $n \times k$ ，其中k是矩阵A的秩（或更具体地说，是奇异值的数量）。
2. 返回的 `s` 是一个包含奇异值的数组，而不是对角矩阵。为了构建对角矩阵 `Sigma`，我们需要创建一个适当大小的零矩阵，并将其对角线上的元素设置为 `s` 中的值。

3. 由于浮点运算的精度问题，直接比较两个矩阵是否相等通常不是一个好主意。相反，我们使用 `np.allclose` 来检查两个矩阵是否足够接近。
4. 在实际应用中，SVD的截断版本（即只保留最大的几个奇异值）经常被用于数据压缩、噪声过滤等任务。这可以通过简单地保留 s 、 u 和 v^T （或 v ）中的前 k 个元素来实现。

3. 楚列斯基分解

楚列斯基分解（Cholesky Decomposition）是线性代数中的一个重要概念，特别是在数值分析和计算科学领域中被广泛应用。楚列斯基分解是指将一个对称正定矩阵 A 表示为 LL^T 的形式，其中 L 是一个下三角矩阵，且其对角线元素都是非负实数。这里， L^T 表示 L 的转置矩阵。这种分解使得我们能够高效地解决线性系统 $Ax = b$ ，而无需直接处理矩阵 A 。

1. **对称正定矩阵**：楚列斯基分解仅适用于对称正定矩阵。对称矩阵意味着 $A = A^T$ ，即矩阵的转置等于其自身。正定矩阵则意味着对于任何非零向量 x ，都有 $x^T Ax > 0$ 。
2. **下三角矩阵**：分解得到的 L 是一个下三角矩阵，即除了对角线及其下方的元素外，其他元素都为 0。
3. **非负对角线元素**： L 的对角线元素都是非负的，这是为了确保 LL^T 的结果是一个正定矩阵。
4. **高效性**：楚列斯基分解在计算上非常高效，特别是对于大型稀疏对称正定矩阵。它允许我们以较低的计算和存储成本解决线性系统。

注意事项

- 在进行楚列斯基分解之前，通常需要检查矩阵是否是对称正定的。不是所有对称矩阵都能进行楚列斯基分解。
- 对于某些特殊类型的对称正定矩阵（如希尔伯特矩阵），可能需要特别的算法来确保计算的稳定性和准确性。

在NumPy中，楚列斯基分解（Cholesky Decomposition）可以通过 `numpy.linalg.cholesky` 函数直接实现。这个函数接受一个对称正定矩阵作为输入，并返回其下三角的Cholesky因子（即 L 矩阵），使得 $L @ L.T$ （ $@$ 是矩阵乘法运算符）等于原始矩阵。

以下是一个使用NumPy实现楚列斯基分解的示例代码：

```

import numpy as np

# 定义一个对称正定矩阵A
A = np.array([[4, 12, -16],
              [12, 37, -43],
              [-16, -43, 98]])

# 检查A是否是对称矩阵（可选步骤，但进行Cholesky分解前通常需要确保这一点）
assert (A == A.T).all(), "Matrix A is not symmetric."

# 使用numpy.linalg.cholesky进行楚列斯基分解
L = np.linalg.cholesky(A)

# 验证分解结果: L @ L.T 应该等于 A
assert np.allclose(L @ L.T, A), "Cholesky decomposition failed."

# 输出L矩阵
print("L矩阵（Cholesky因子）:")
print(L)

```

在这个例子中，我们首先定义了一个对称正定矩阵 A 。然后，我们使用 `numpy.linalg.cholesky` 函数来计算其Cholesky因子 L 。之后，我们通过检查 $L @ L.T$ 是否等于原始矩阵 A 来验证分解的正确性。最后，我们输出了 L 矩阵。

请注意，在实际应用中，你可能不需要显式地检查矩阵是否对称，因为如果你确定你的矩阵是对称正定的，那么就可以直接使用 `numpy.linalg.cholesky` 函数。然而，如果矩阵可能不是对称的，或者你不确定它是否正定，那么在调用 `numpy.linalg.cholesky` 之前进行检查是一个好习惯。

另外，需要注意的是，`numpy.linalg.cholesky` 函数只返回下三角的Cholesky因子 L ，而不返回 L 的转置或完整的分解形式 $L L^T$ 。但是，由于 L 是下三角的，你可以很容易地通过计算 $L @ L.T$ 来得到原始矩阵 A 。

4. QR分解

QR分解（QR Decomposition），也被称为正交三角分解法，是一种将矩阵分解为一个正交矩阵 Q 和一个上三角矩阵 R 的乘积的矩阵分解方法。对于任意实（或复）矩阵 A ，如果它能够被分解为一个正交（或酉）矩阵 Q 和一个上三角矩阵 R 的乘积，即 $A = QR$ ，则称这种分解为 A 的QR分解。

- **正交矩阵 Q** : Q 是一个 $m \times m$ 的正交矩阵（对于复数矩阵，则是酉矩阵），满足 $Q^T * Q = I$ （或 $Q^H * Q = I$ ，其中 H 表示共轭转置， I 是单位矩阵）。正交矩阵的列向量是单位向量且两两正交。

- **上三角矩阵R**: R是一个 $m \times n$ 的上三角矩阵, 其对角线以下的元素全为0 (对于方阵, 则是一个上三角矩阵; 对于非方阵, R的右侧可能会有一些行是全0的)。
- **适用性广**: QR分解适用于对称矩阵和非对称矩阵, 也适用于实矩阵和复矩阵。
- **计算特征值**: QR分解是求一般矩阵全部特征值的最有效并广泛应用的方法之一。一般矩阵先经过正交相似变化成为Hessenberg矩阵, 然后再应用QR方法求特征值和特征向量。
- **线性最小二乘**: QR分解被广泛应用于线性最小二乘问题的求解, 通过将问题转化为上三角矩阵的求解, 可以简化计算过程。

QR分解有多种算法实现, 包括但不限于Gram-Schmidt算法、Householder算法和Givens算法。这些算法各有特点, 但目标都是将矩阵A分解为一个正交矩阵Q和一个上三角矩阵R的乘积。

- **Gram-Schmidt算法**: 通过一系列的投影操作, 将A的列向量转化为正交向量, 从而构造出Q和R。
- **Householder算法**: 利用Householder反射矩阵, 通过一系列的反射操作, 将A转化为上三角矩阵R, 同时构造出正交矩阵Q。
- **Givens算法**: 通过一系列的旋转操作, 逐步将A的下三角元素化为0, 同时保持上三角元素不变, 从而得到R, 并构造出Q。

在NumPy中, QR分解可以通过 `numpy.linalg.qr` 函数直接实现, 而不需要手动编写QR分解的算法。 `numpy.linalg.qr` 函数内部已经实现了这些算法中的一种 (通常是Householder算法, 因为它在数值上更稳定), 并为用户提供了方便的接口。

以下是使用NumPy进行QR分解的示例代码:

```

import numpy as np

# 定义一个矩阵A
A = np.array([[12, -51, 4],
              [6, 167, -68],
              [-4, 24, -41]])

# 使用numpy.linalg.qr进行QR分解
Q, R = np.linalg.qr(A)

# Q是一个正交矩阵（或酉矩阵，对于复数矩阵），R是一个上三角矩阵
# 验证QR分解：Q @ R 应该等于原始矩阵A（由于浮点运算的精度问题，使用np.allclose进行比较）
assert np.allclose(Q @ R, A), "QR decomposition failed."

# 输出Q和R
print("Q矩阵（正交矩阵）:")
print(Q)
print("R矩阵（上三角矩阵）:")
print(R)

```

在这个例子中，`numpy.linalg.qr` 函数返回了两个矩阵：Q 和 R。Q 是一个正交矩阵（其列向量是单位向量且两两正交），而 R 是一个上三角矩阵。这两个矩阵的乘积 $Q @ R$ 应该等于原始矩阵 A（由于浮点运算的精度问题，我们通常不会直接比较它们是否完全相等，而是使用 `np.allclose` 来检查它们是否足够接近）。

需要注意的是，`numpy.linalg.qr` 函数还可以接受一个 `mode` 参数，该参数控制函数的输出。默认情况下（`mode='reduced'`），Q 的形状是 (m, k) ，其中 m 是 A 的行数， k 是 A 的列数（如果 A 是方阵，则 k 等于 m ；如果 A 是瘦矩阵，则 k 小于 m ），而 R 的形状是 (k, n) ，其中 n 是 A 的列数。如果 `mode='complete'`，则 Q 的形状是 (m, m) ，并且 R 的底部会包含一些额外的行（对于非方阵 A，这些行将是全零的），以便 Q 成为一个完整的正交矩阵。然而，在大多数情况下，默认的 `mode='reduced'` 就足够了。

1.4 Numpy与多项式运算

1.4.1 多项式的构造与运算

`numpy.poly1d` 是NumPy中用于表示一维多项式的类。你可以通过系数来创建多项式，并使用这个类进行多项式的计算（如加法、乘法、求值等）。

```
import numpy as np

# 创建一个多项式  $2x^2 + 3x + 1$ 
p = np.poly1d([2, 3, 1])

# 显示多项式
print(p)

# 计算多项式在x=5的值
print(p(5))

# 多项式的加法
q = np.poly1d([1, 2]) #  $x + 2$ 
print(p + q)

# 多项式的乘法
print(p * q)
```

使用 `numpy.poly1d` 创建的多项式对象可以直接进行四则运算（加、减、乘、除），这些操作会返回新的 `numpy.poly1d` 对象，表示结果多项式。

下面是如何使用 `numpy.poly1d` 进行多项式四则运算的示例：

```

import numpy as np

# 创建两个多项式
p = np.poly1d([2, 3, 1]) # 表示  $2x^2 + 3x + 1$ 
q = np.poly1d([1, 2])    # 表示  $x + 2$ 

# 加法
p_plus_q = p + q
print("p + q =", p_plus_q)

# 减法
p_minus_q = p - q
print("p - q =", p_minus_q)

# 乘法
p_times_q = p * q
print("p * q =", p_times_q)

# 注意：除法在多项式运算中比较特殊，因为除法的结果可能包括商和余数
# numpy.poly1d 提供了 divmod 函数，用于同时返回商和余数
q_div_p, remainder = np.poly1d.divmod(q, p)
print("q / p (商) =", q_div_p)
print("余数 =", remainder)

# 如果你只想要商（不考虑余数），可以使用 div 函数（但在 numpy.poly1d 中没有直接的 div 方法，
# 你需要手动计算或使用 divmod 并只取第一个返回值）
# 注意：这里我们使用 divmod 并只取商
q_div_p_only_quotient = np.poly1d.divmod(q, p)[0]
print("q / p (仅商) =", q_div_p_only_quotient)

# 但是，通常对于多项式除法，我们更关心的是商，因此可以像上面那样处理

```

多项式除法的结果可能不是整数或简单的有理数，而是一个新的多项式（商）和一个余数多项式。在上面的代码中，`np.poly1d.divmod(q, p)` 同时返回了商和余数。如果你只想要商，可以像示例中那样只取 `divmod` 返回值的第一个元素。

1.4.2 多项式上的复杂运算

`np.poly1d` 对象提供了直接进行微分（求导）和积分运算的方法。这些方法分别是 `.deriv()` 和 `.integ()`。

微分（求导）使用 `.deriv(m=1, scale=1, rcond=None)` 方法可以对多项式进行微分。其中，`m` 指定了微分的阶数（默认为1），`scale` 是一个可选参数，用于在微分后调整多项式的系数（默认为1，即不调整），`rcond` 在这个上下文中不适用，因为 `np.poly1d` 的微分不涉及任何数值稳定性问题。

```
import numpy as np

# 创建一个多项式
p = np.poly1d([2, 3, 1]) # 表示 2x^2 + 3x + 1

# 对多项式进行一阶微分
dp = p.deriv()
print("一阶微分:", dp)

# 对多项式进行二阶微分
ddp = p.deriv(2)
print("二阶微分:", ddp)
```

积分使用 `.integ(m=1, k=0)` 方法可以对多项式进行积分。其中，`m` 指定了积分的阶数（默认为1），`k` 是积分常数，用于在每次积分后添加（默认为0）。

```
# 对多项式进行一阶积分
ip = p.integ()
print("一阶积分 (积分常数为0):", ip)

# 对多项式进行一阶积分，并指定积分常数为1
ip_k1 = p.integ(k=1)
print("一阶积分 (积分常数为1):", ip_k1)

# 注意：连续积分时，积分常数会累加
ipp = p.integ(2) # 这相当于两次应用.integ(k=0)
print("二阶积分 (积分常数为0):", ipp)

ipp_k2 = p.integ(2, k=2) # 这相当于两次应用.integ()，但总积分常数为2
print("二阶积分 (总积分常数为2):", ipp_k2)
```

请注意，在连续进行积分时，如果每次积分都指定了非零的积分常数，那么这些常数会累加。然而，在实际应用中，通常只在第一次积分时指定一个积分常数，而在后续的积分中保持该常数为零（除非有特定的理由需要改变它）。

此外，由于 `np.poly1d` 的积分和微分都是符号计算，因此它们不会产生数值误差（除非在后续的计算中使用了浮点数进行求值）。

`np.poly1d` 对象本身并没有直接提供求多项式根的方法，因为这是一个表示多项式的类，而不是进行符号数学计算的库。然而，你可以使用 NumPy 的 `numpy.roots` 函数来求取一个多项式（以系数形式给出）的根。

`numpy.roots` 函数接受一个包含多项式系数的数组（从最高次项到常数项），并返回多项式的根。这些根可能是实数或复数，具体取决于多项式。

下面是如何使用 `numpy.roots` 来求取由 `np.poly1d` 表示的多项式的根的示例：

```
import numpy as np

# 创建一个多项式
p = np.poly1d([2, 3, 1]) # 表示 2x^2 + 3x + 1

# 获取多项式的系数
coeffs = p.coef

# 使用 numpy.roots 求多项式的根
roots = np.roots(coeffs)

print("多项式的根:", roots)
```

在这个例子中，`p.coef` 返回了一个包含多项式系数的数组（在这个例子中是 `[2, 3, 1]`）。然后，我们将这个系数数组传递给 `numpy.roots` 函数，它返回了多项式的根。

需要注意的是，`numpy.roots` 函数返回的根是按照它们在复数平面上的位置来排序的，但这并不总是意味着它们按照模的大小或实部和虚部的某种顺序来排序。此外，如果多项式有重根，`numpy.roots` 也会返回相应的多个根（每个根都表示一次）。

还要注意的，由于浮点数运算的精度限制，对于某些多项式（特别是高次多项式或具有复杂系数的多项式），`numpy.roots` 返回的根可能只是近似值，而不是精确值。因此，在需要高精度解的情况下，可能需要使用专门的符号数学库（如 SymPy）来求解多项式的根。

这里我可以补充一种如何从0开始计算一个方程数值解（不使用任何库函数）的方法。Newton法，也称为切线法，是一种寻找函数零点的有效方法。其思路是从一个初始估计值开始，不断迭代更新，直到找到零点或达到预定精度。

假设我们想要求解方程 $y = x^3 - x - 1$ 的零点，我们从一个初始点 x_0 开始。Newton法首先在 $x = x_0$ 处求函数的切线，并找到切线与x轴的交点 x_1 ，然后在 $x = x_1$ 处再次求切线，找到新的交点 x_2 ，如此不断迭代下去，最终会逼近 x_0 附近的零点。以下是用Python实现的 Newton法示例代码：


```

import numpy as np
def f(x):
    y=x**3-x-1#求根方程的表达式
    return y
def g(x):
    y=3*x**2-1#求根方程的导函数
    return y
def main():
    x_0=1.5 #取初值
    e=10**(-9) #误差要求
    L=0 #初始化迭代次数
    while L<100: #采用残差来判断
        x1=x_0-f(x_0)/g(x_0) #迭代公式,x(n+1)=x(n)-f(x(n))/f'(x(n))
        x_0=x1
        L=L+1 #统计迭代次数
        if abs(f(x_0)-0)<e:
            break
    print(f"x1={x1}") #输出数值解
    print(f(x_0)-0) # 验证解的正确性
    print(f"L={L}") #输出迭代次数
if __name__ == '__main__':
    main()

```

注意：Python中不需要像MATLAB那样使用特殊函数来控制结果的精度，因为Python的浮点数运算本身就具有较高的精度。

最终解得：

```
x1=1.3247179572447898 1.865174681370263e-13 L=4
```

程序能够利用 Newton法搜索到起始点最近的一个零点。

`numpy.polyfit` 函数用于拟合一个多项式 $p(x) = p[0] * x^{deg} + p[1] * x^{(deg-1)} + \dots + p[deg]$ 到给定的数据点，其中 `deg` 是多项式的度数。该函数返回多项式的系数，从最高次项到常数项。

下面是一个使用 `numpy.polyfit` 进行多项式拟合的详细示例：

```

import numpy as np
import matplotlib.pyplot as plt

# 示例数据: x值和对应的y值 (可能含有噪声)
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 0.8, 0.9, 0.1, -0.8, -1])

# 指定多项式的度数
deg = 2

# 使用numpy.polyfit进行多项式拟合
# 返回的coeffs是多项式的系数, 从最高次项到常数项
coeffs = np.polyfit(x, y, deg)

# 使用numpy.poly1d创建多项式对象 (可选)
p = np.poly1d(coeffs)

# 打印拟合得到的多项式
print("拟合得到的多项式:", p)

# 使用拟合得到的多项式进行预测 (可选)
x_fit = np.linspace(min(x), max(x), 100) # 生成一系列x值用于绘图
y_fit = p(x_fit) # 计算对应的y值

# 绘图展示原始数据和拟合曲线
plt.scatter(x, y, label='原始数据')
plt.plot(x_fit, y_fit, color='red', label='拟合曲线')
plt.xlabel('x')
plt.ylabel('y')
plt.title('多项式拟合示例')
plt.legend()
plt.show()

```

在这个例子中, 我们首先定义了x和y的示例数据 (这里y值是模拟的, 可能包含噪声)。然后, 我们指定了多项式的度数 `deg`, 并使用 `numpy.polyfit` 函数对x和y进行拟合, 得到了多项式的系数 `coeffs`。接着, 我们使用 `np.poly1d` (这是可选的) 创建了一个多项式对象 `p`, 它可以更方便地用于计算多项式的值或进行其他多项式运算。最后, 我们使用拟合得到的多项式对一系列x值进行预测, 并绘制了原始数据点和拟合曲线。

需要注意的是, 多项式拟合的度数 `deg` 是一个重要的参数, 它会影响拟合的效果。如果度数太低, 可能无法很好地拟合数据的复杂变化; 如果度数太高, 则可能会出现过拟合现象, 即模型在训练数据上表现

得很好，但在新的、未见过的数据上表现不佳。因此，在实际应用中，通常需要通过交叉验证等方法来选择合适的度数。

扩展阅读

Numpy这方面的详细教程，在Datawhale有人做过：[巨硬的Numpy](#)